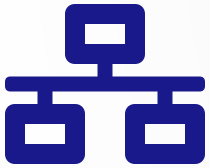


Week 8: Web Crawling & Indexing Architecture

Building the Infrastructure of Search

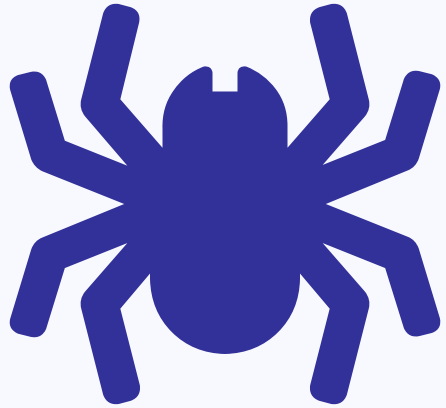
Course Plan

Week 1	✓	Introduction to IR
Week 2	✓	Text Analysis & Preprocessing
Week 3	✓	Vector Space Model
Week 4	✓	Index Construction
Week 5	✓	Retrieval Models
Week 6	✓	Evaluation in IR
Week 7	✓	Link Analysis (PageRank)
Week 8		**Web Crawling & Architecture**
Week 9		Clustering & Classification
Week 10		Learning to Rank
Week 11		Advanced IR (NER)
Week 12		Semantic Search

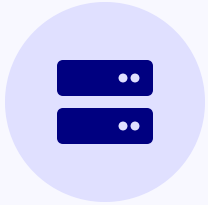
What is Web Crawling?

Web Crawling is the process of systematically browsing the World Wide Web, typically for the purpose of **Web Indexing**.

- Discover new pages via links (HTML anchor tags).
- Download content for processing and storage.
- Periodically revisit pages to check for updates.
- Also known as: *Spiders, Bots, Harvesters*.



Challenges at Web Scale



Sheer Scale

The web contains billions of documents and petabytes of data.

> 50 Billion Indexed Pages

Requires distributed crawling, storage, and indexing systems.

Rate of Change

Content is constantly updated, added, or deleted.

Real-time News & Social

Requires sophisticated recrawling policies to maintain freshness.

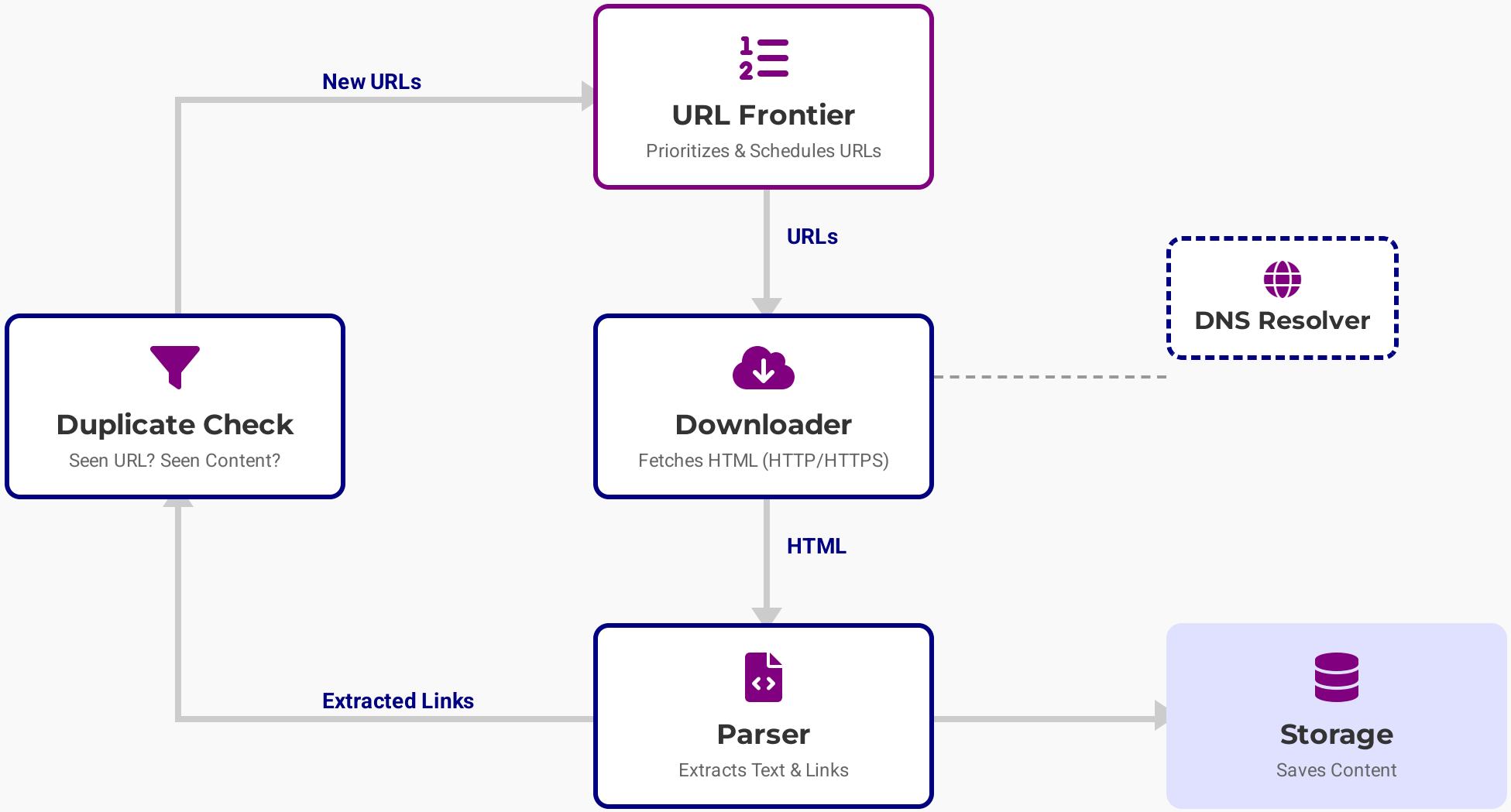
Diversity & Quality

Various formats (HTML, PDF, Images), languages, and encodings.

Spam & Adversarial Web

Requires robust parsing and spam detection algorithms.

High-Level Architecture



The URL Frontier

The "To-Do" List

The central data structure that manages all URLs waiting to be crawled.

Prioritization

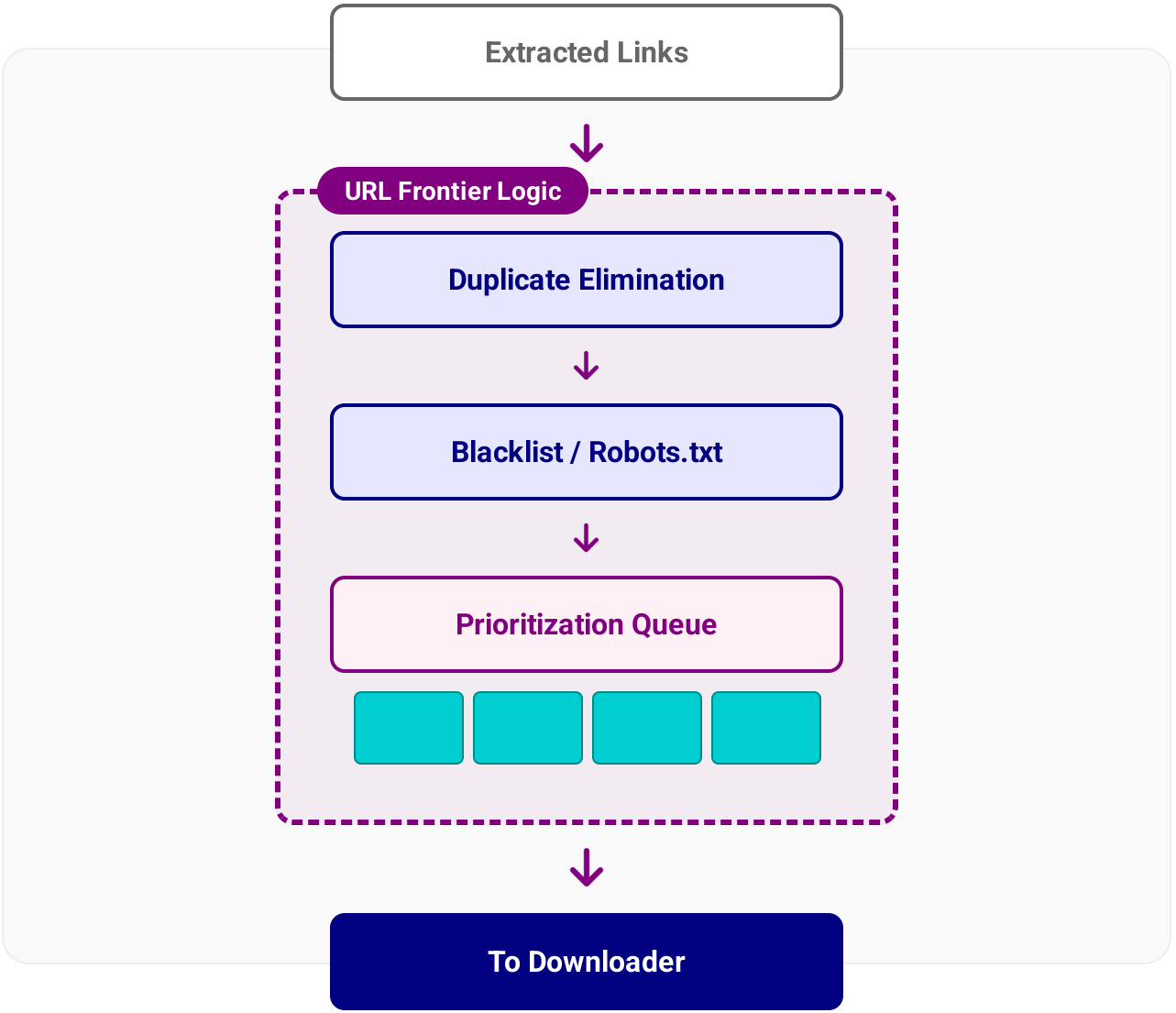
Deciding *which* page is most important to fetch next (e.g., high PageRank).

Politeness

Deciding *when* to fetch a page to avoid overloading a specific server.

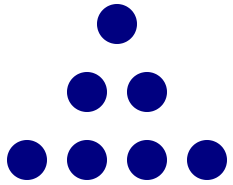
Freshness

Scheduling re-crawls for pages that change frequently.



Prioritization Policies

Breadth-First Search (BFS)



Level-by-Level

The crawler visits pages in the order they are discovered. It finishes all links at depth *d* before moving to depth *d+1*.

PRO Easy to implement (FIFO Queue).

CON May crawl low-quality pages before important ones deep in the site.

Importance-Based (PageRank)

Prioritize High Value

The crawler estimates the importance of a URL (using In-degree or partial PageRank) and prioritizes the queue accordingly.

PRO Discovers high-quality content faster.

CON Computationally expensive; requires maintaining a priority queue.

Politeness & Ethics

> **The Golden Rule:** A crawler must never degrade the performance of the website it visits. It should be a "good citizen" of the web.

Respect Frequency

Wait between requests to the same domain (Crawl-Delay).
Standard: 1 request every few seconds.

Identify Yourself

Send a valid **User-Agent** string with contact info (email/URL) so admins can contact you if needed.

Respect Exclusion

Always check and obey **robots.txt**. Do not crawl disallowed paths or private data.

⚠ Consequence: Aggressive crawling is indistinguishable from a Denial of Service (DoS) attack and will get your IP banned.

Robots Exclusion Protocol

Location

Must be placed at the **root** of the domain.

Example: `http://example.com/robots.txt`

Directives

User-agent: Specifies which bot the rule applies to.

Disallow: URL prefix to block.

Allow: Exception to a disallow rule.

Sitemap: Location of the XML sitemap.

Limitations

It is advisory, not enforceable (bad bots ignore it).

It does not hide data (files are still public).

robots.txt

```
# Rules for all bots
User-agent: *
Disallow: /admin/
Disallow: /private/

# Specific rules for Googlebot
User-agent: Googlebot
Allow: /private/public-page.html

# Crawl delay (seconds)
Crawl-delay: 10

Sitemap: http://example.com/sitemap.xml
```

XML Sitemaps

A Roadmap for Crawlers

An XML file that lists URLs for a site along with additional metadata. It helps crawlers discover pages that might be isolated or deep in the site hierarchy.

Key Metadata

`<loc>` : The absolute URL of the page.

`<lastmod>` : Date of last modification. Critical for freshness.

`<changefreq>` : Hint on how often it changes (e.g., daily).

`<priority>` : Relative importance (0.0 to 1.0).

> **Pro Tip:** Submit your sitemap directly to search engines via their webmaster tools (e.g., Google Search Console) to ensure immediate discovery.

sitemap.xml

```xml
<urlset xmlns="http://www.sitemaps.org/ ... ">
<url>
<loc>http://www.example.com/</loc>
<lastmod>2023-10-01</lastmod>
<changefreq>daily</changefreq>
<priority>1.0</priority>
</url>
<url>
<loc>http://www.example.com/about</loc>
 <lastmod>2023-09-15</lastmod>
 <changefreq>monthly</changefreq>
 <priority>0.8</priority>
 </url>
 </urlset>
```

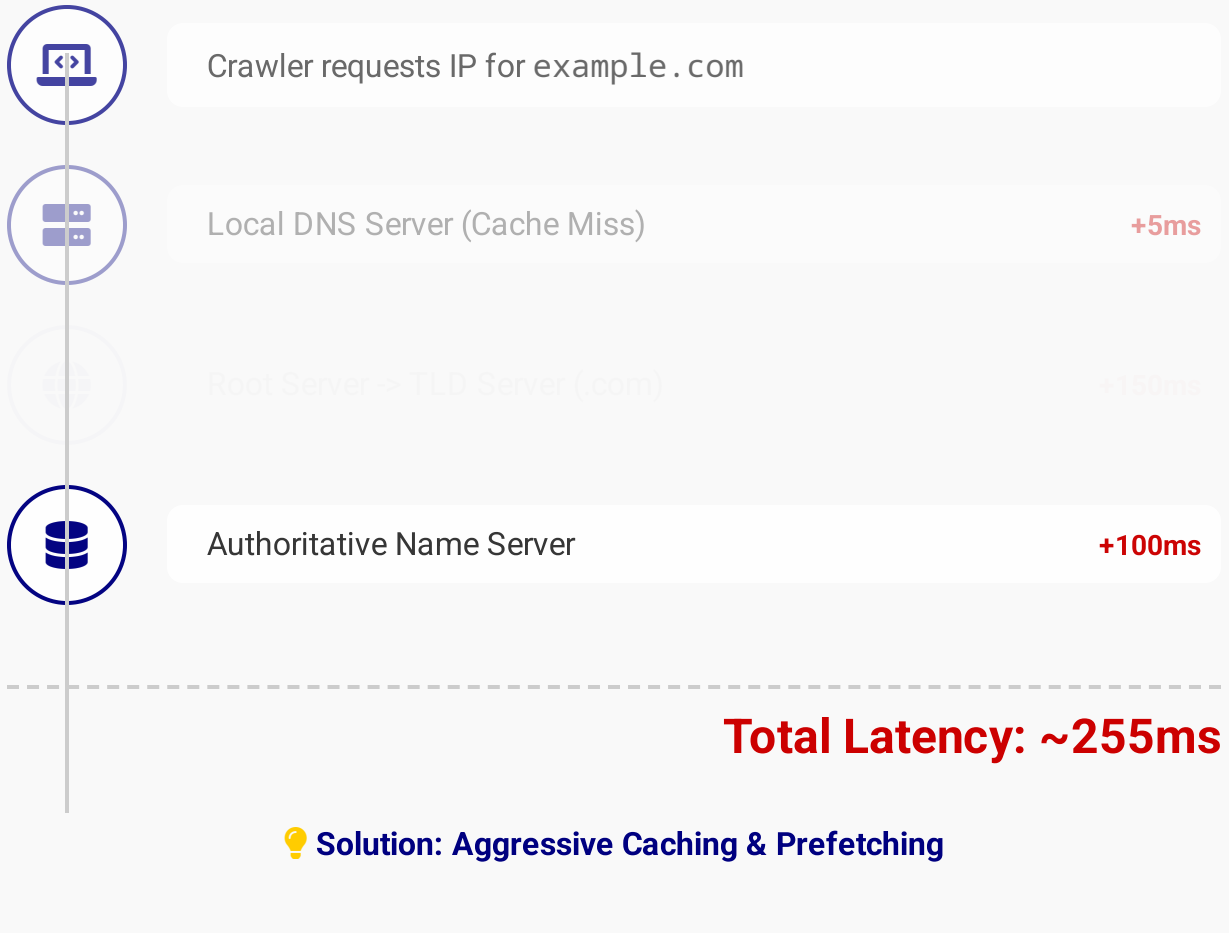
DNS Resolution Bottleneck

The Latency Trap

Before fetching a page, the crawler must translate the hostname (e.g., `www.wikipedia.org`) into an IP address.

Standard DNS lookups are **synchronous** and involve multiple network round-trips.

> **Impact:** Even with a fast connection, DNS can add **20ms to 500ms** per URL. At web scale (1B pages), this is years of waiting time.



DNS Caching Strategies

Local Caching

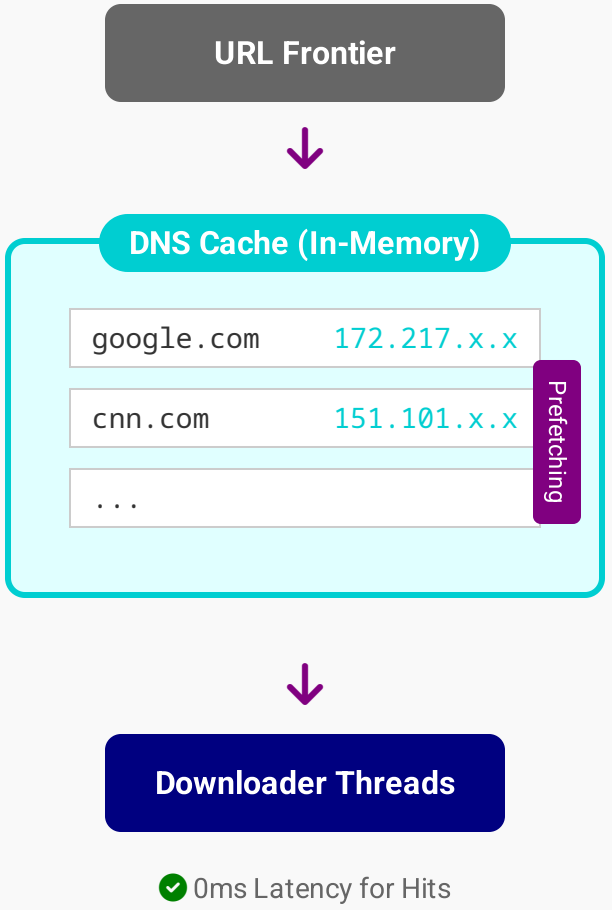
Maintain a large, in-memory LRU cache of IP addresses. Avoids repeated network lookups for the same domain.

Prefetching

Resolve hostnames *before* the downloader requests them. A separate thread scans the queue and populates the cache.

Batch Resolution

Send DNS requests in parallel batches (e.g., using UDP) to maximize throughput and hide latency.



Distributed Crawling

Why Distribute?

A single machine is limited by bandwidth, storage I/O, and CPU. To crawl billions of pages, we need a cluster.

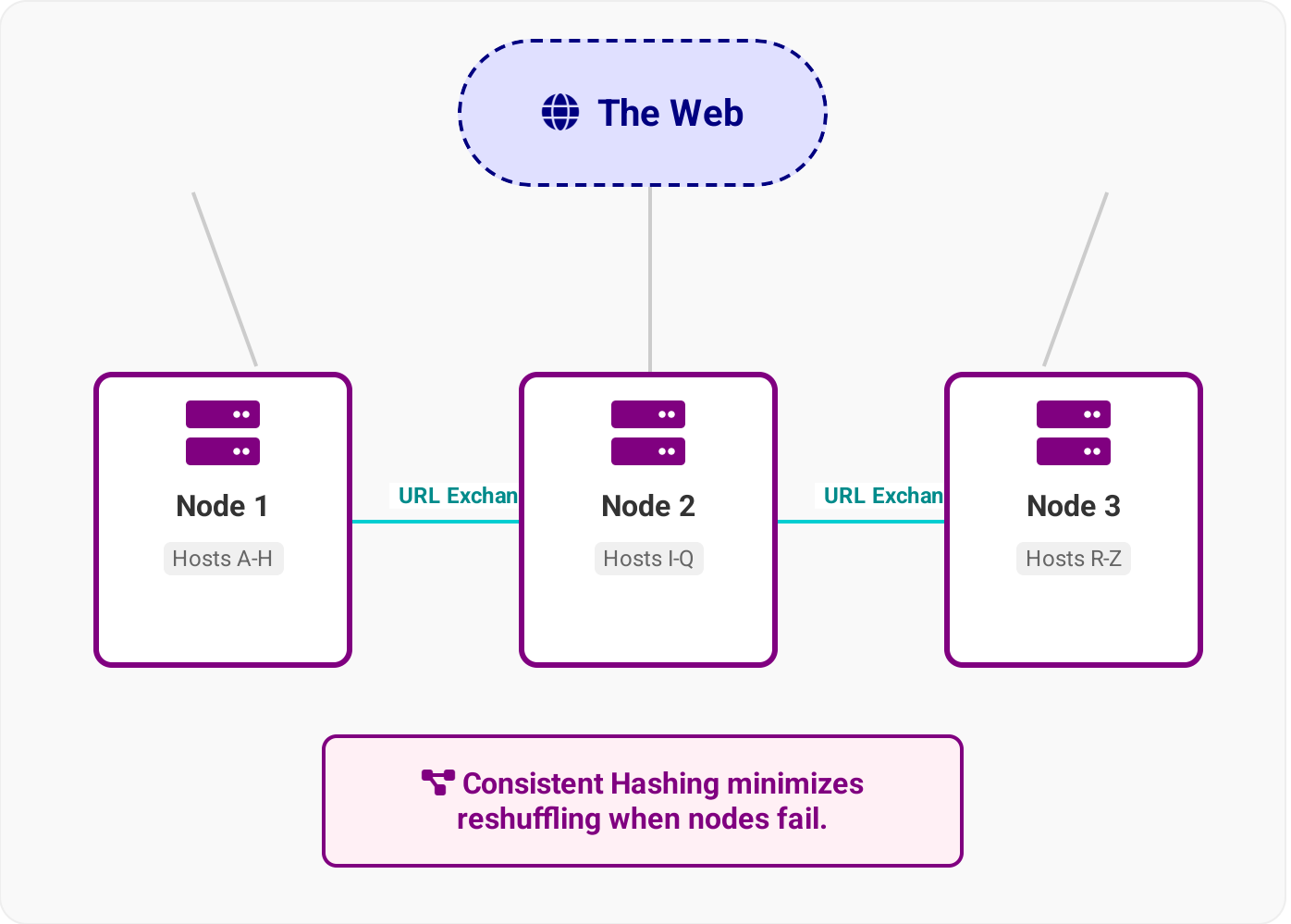
Partitioning Strategy

How do we split the work?

- **By Hostname:** `hash(host) % N`. Ensures all URLs from *cnn.com* go to the same node.
- **Politeness:** Easier to manage crawl delays if one node owns the host.

URL Exchange

If Node A crawls a page with a link to a host owned by Node B, it must transfer that URL.



Consistent Hashing

The Problem

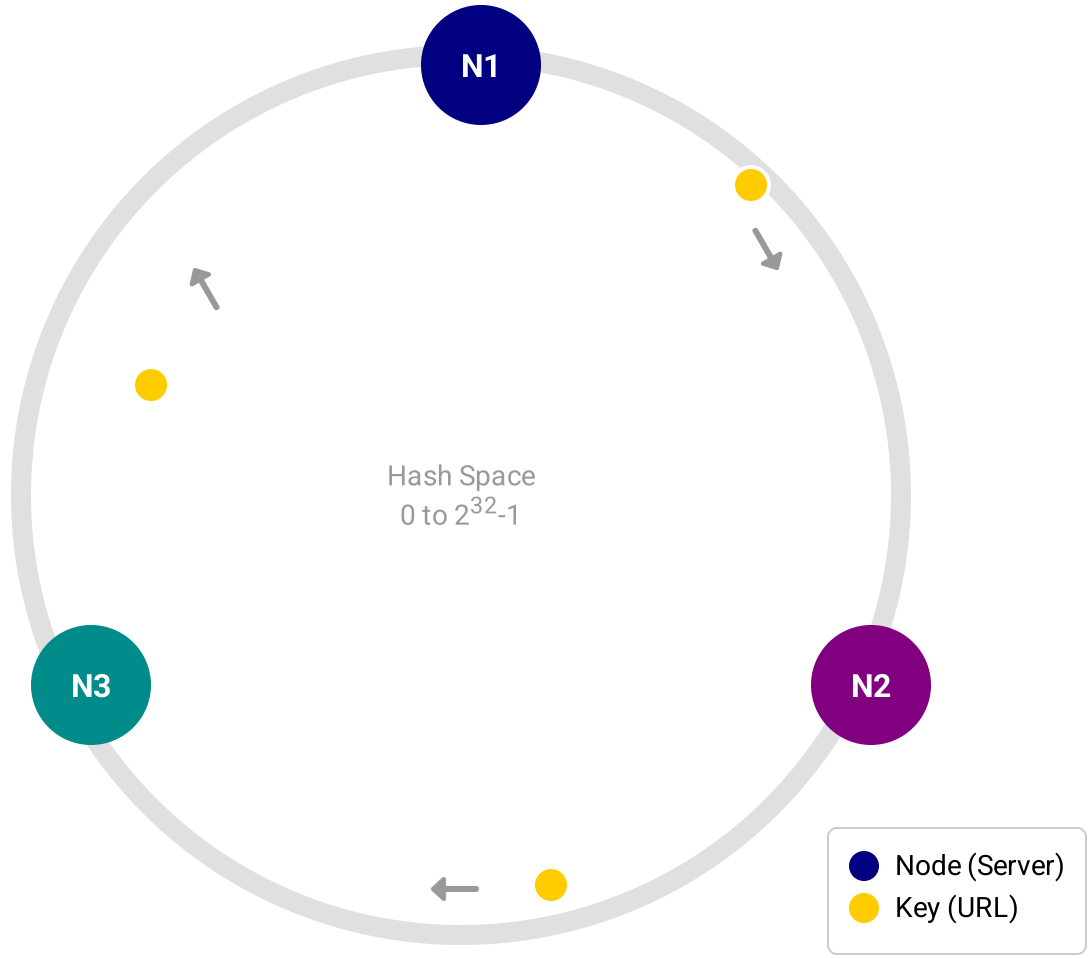
With standard `hash(url) % N`, adding or removing a node changes N, causing nearly **all** URLs to be remapped to different servers.

The Solution

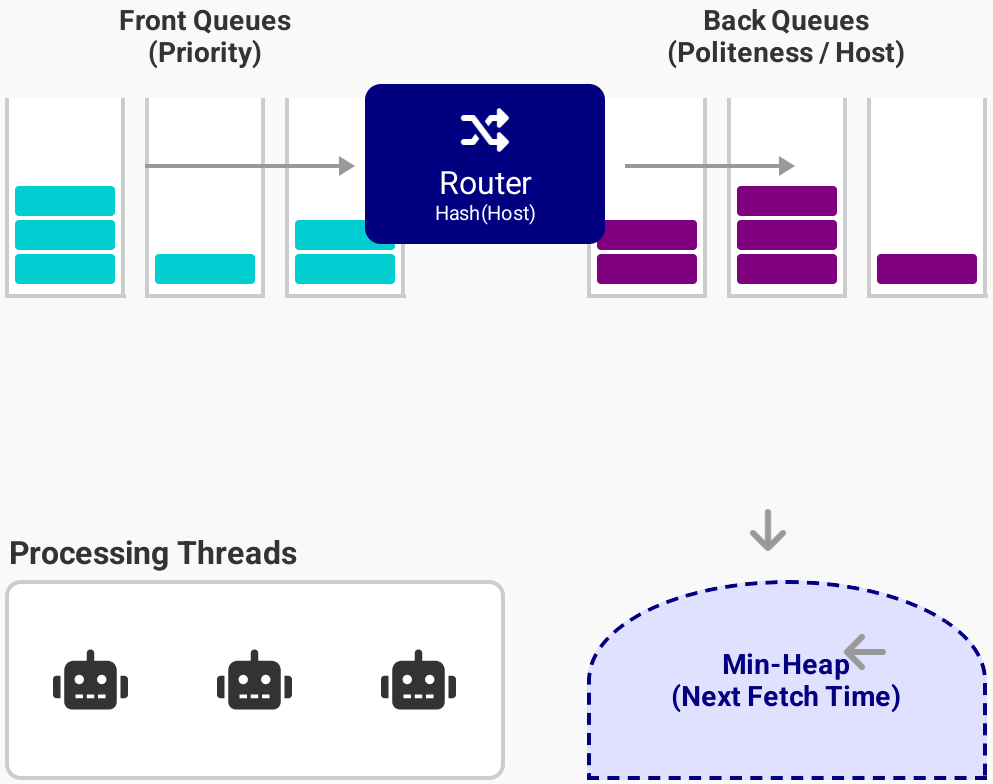
Map both **Nodes** and **URLs** to a circular space (Ring).
A URL is assigned to the first Node encountered moving **clockwise**.

Benefit

Adding/Removing a node only affects its immediate neighbors. Minimal data movement.



Mercator Crawler Model



Decoupled Logic

Mercator separates **Prioritization** (Front Queues) from **Politeness** (Back Queues).

Front Queues (Priority)

URLs are added to a front queue based on importance (e.g., PageRank). A biased selector picks from these queues (high priority more often).

Back Queues (Politeness)

Each back queue belongs to a single host. The heap ensures we only pull from a queue when its **Crawl-Delay** has passed.

Source: Heydon & Najork, "Mercator: A scalable, extensible web crawler" (1999)

Duplicate Detection

The Problem: Approximately **30-40%** of web pages are duplicates or near-duplicates.

Wastes crawl bandwidth, storage, and frustrates users.

Exact Duplicates

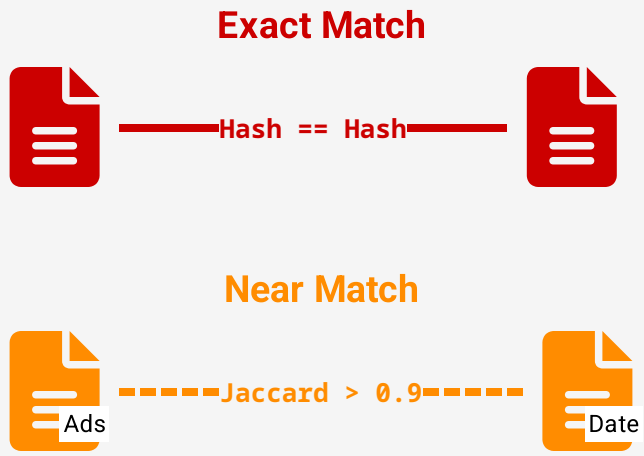
Identical content, different URLs (e.g., mirrors, session IDs, tracking parameters).

`MD5 Checksum` `SHA-1`

Near-Duplicates

Same content with minor differences (e.g., timestamps, dynamic ads, navigation menus).

`Shingling` `MinHash`



Shingling & Jaccard Similarity

1. k-Shingling (n-grams)

Convert a document into a set of fixed-size substrings.

```
"a rose is a rose is a rose"
```

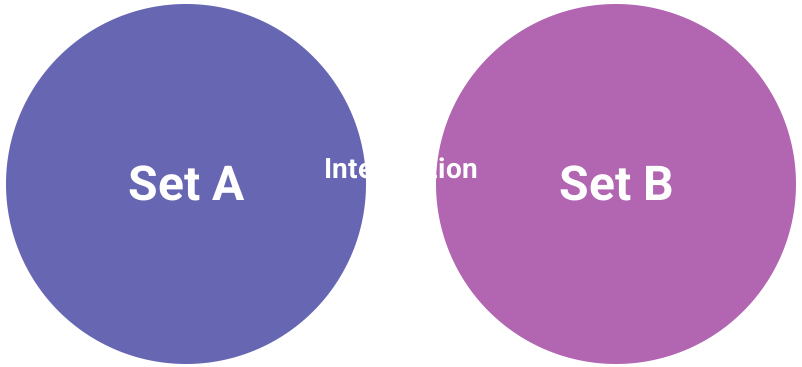
↓

k = 4 (4-grams)

`"a rose is a"` `"rose is a rose"` `"is a rose is"`

* Note: The set removes duplicates. Original text had repetitions, but the set captures the unique patterns.

2. Jaccard Similarity



$$J(A, B) = |A \cap B| / |A \cup B|$$

Intersection over Union

Application: If J(A, B) > 0.9, the documents are considered **Near-Duplicates**.

MinHash Algorithm

The Goal

Compress large sets (documents) into small **signatures** while preserving the ability to estimate Jaccard similarity.

The Mechanism

Apply *k* random hash functions (permutations) to the rows. For each function, the signature value is the index of the **first row** with a '1'.

MinHash Property:

The probability that the MinHash values of two sets are equal is exactly their Jaccard similarity.

$$P(h(A) = h(B)) = J(A, B)$$

Input (Shingles x Docs)

	D1	D2	D3
S1	1	0	1
S2	0	1	0
S3	1	1	0
S4	1	0	1
S5	0	1	0
...	...	...	...

Sparse, High Dimensional

Signature Matrix (Hash x Docs)

	D1	D2	D3
h1	1	2	1
h2	3	2	4
h3	1	3	1

Dense, Low Dimensional

Dimensionality Reduction: 10,000 rows → 100 rows

Example: For h1, D1 has '1' at row 1. D2 has '1' at row 2. D3 has '1' at row 1.
Sig(D1) = 1, Sig(D2) = 2, Sig(D3) = 1.

Locality Sensitive Hashing (LSH)

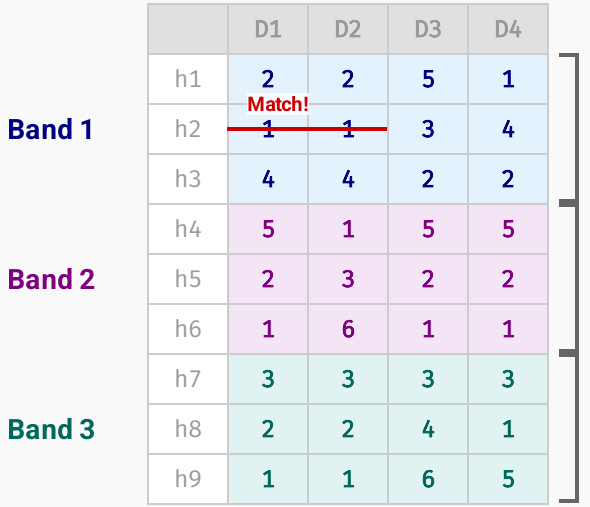
The Bottleneck

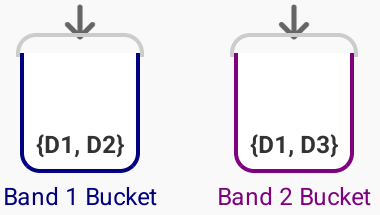
Even with MinHash signatures, comparing every pair of documents is $O(N^2)$. For 10M docs, that's 100 trillion comparisons.

The Solution: Banding

Divide the signature matrix into **b** bands of **r** rows. Hash each band to a bucket.

> **Candidate Pair:** If two documents hash to the same bucket in *at least one* band, they are candidates for full comparison.





D1 & D2 are candidates (Band 1 match).
D1 & D3 are candidates (Band 2 match).

Freshness vs. Age

Freshness

A binary metric. Is the cached copy identical to the live web page?

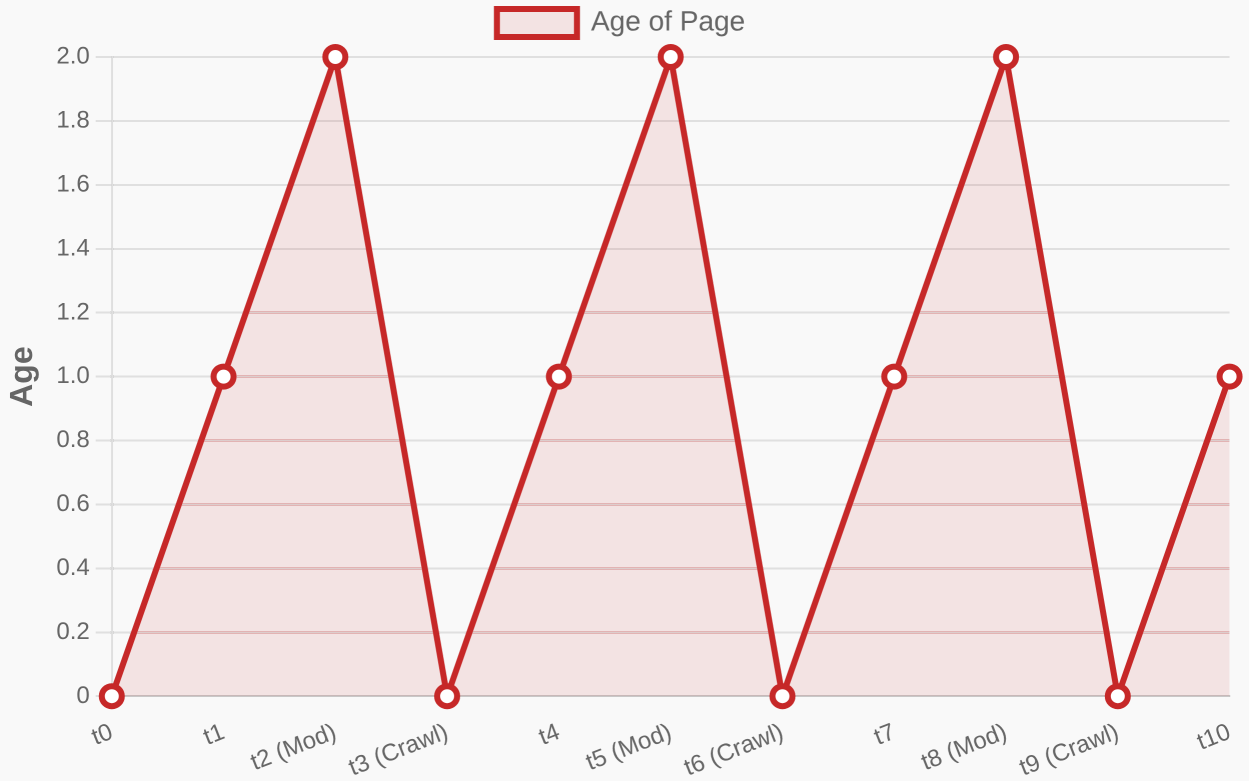
$$F(p, t) = 1 \text{ if fresh, } 0 \text{ if stale}$$

Age

A continuous metric. How much time has passed since the page changed?

$$Age(p, t) = t - modification\_time$$

◎ Goal: Minimize Average Age

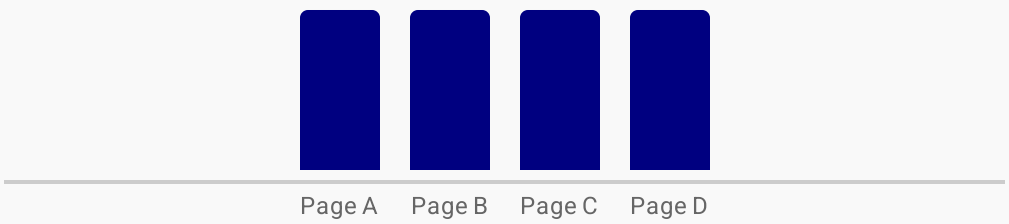


Evolution of "Age" over time. Age increases linearly until a re-crawl (or modification) resets it.

Recrawling Strategies

Uniform Policy

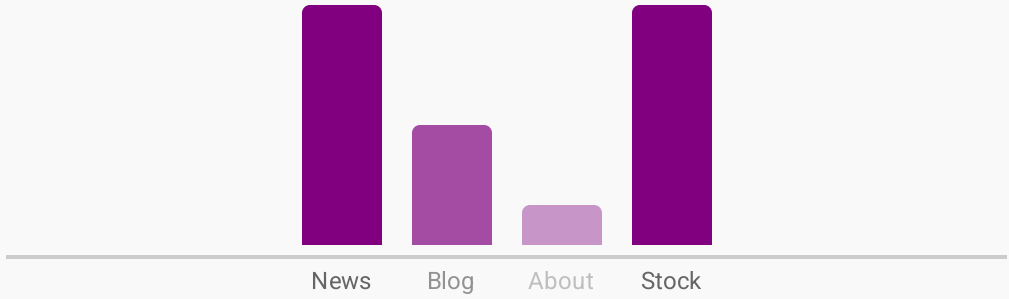
Re-visit every page at the same fixed interval (e.g., every 7 days).



- ✓ **Simple:** Easy to implement.
- ✗ **Inefficient:** Wastes resources on static pages; misses updates on dynamic ones.

Proportional Policy

Re-visit frequency is proportional to the estimated change frequency (λ).



- ✓ **Optimal:** Maximizes average freshness.
- ⚠ **Complex:** Requires estimating λ (Poisson process) for each page.

Key Insight: Cho and Garcia-Molina (2000) proved that proportional allocation is superior, but optimal allocation actually penalizes pages that change *too* frequently (if they change faster than you can crawl, don't bother trying to keep up perfectly).

The Deep Web

The Definition

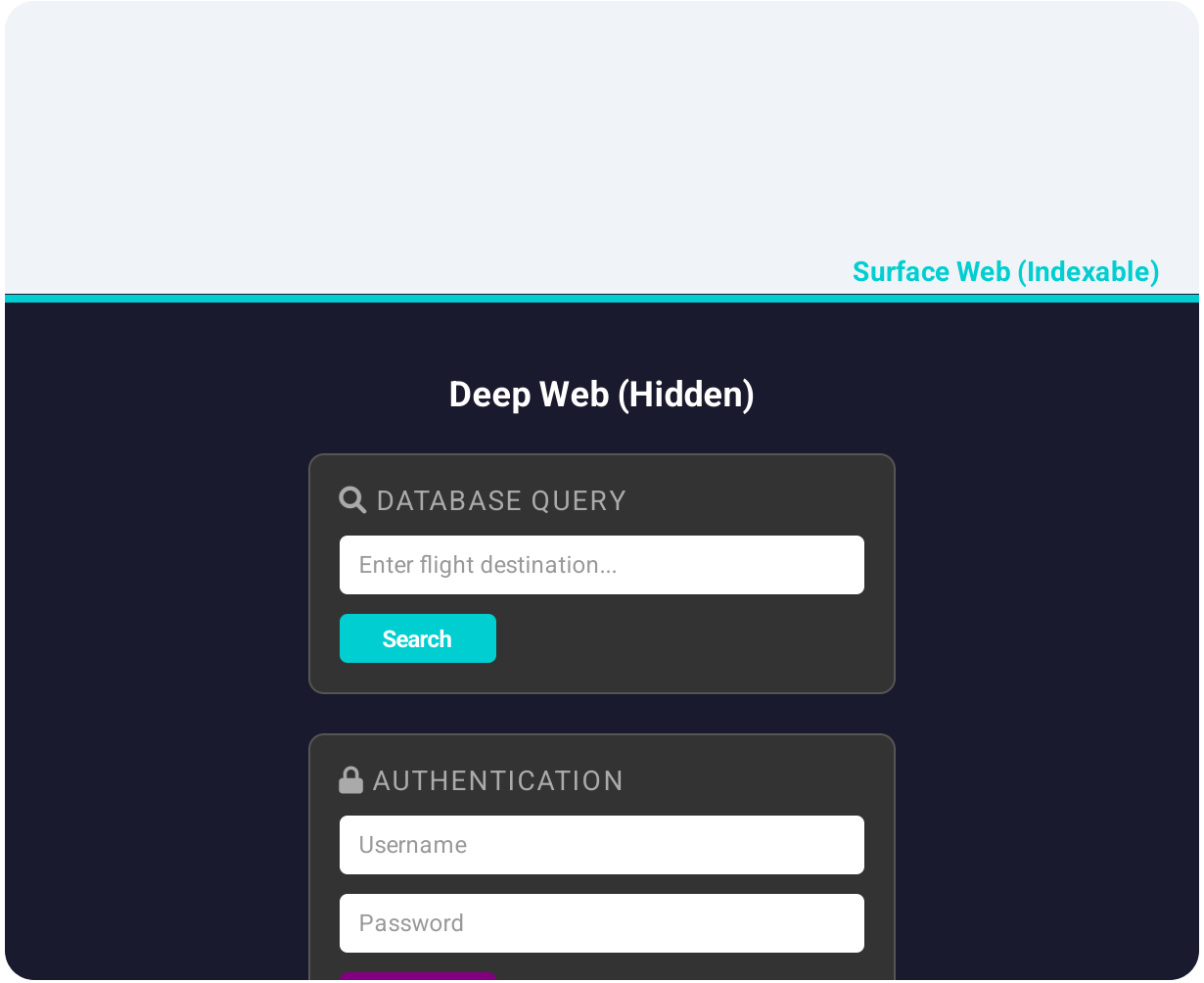
Content that is not part of the Surface Web and cannot be indexed by following static hyperlinks.

The Scale

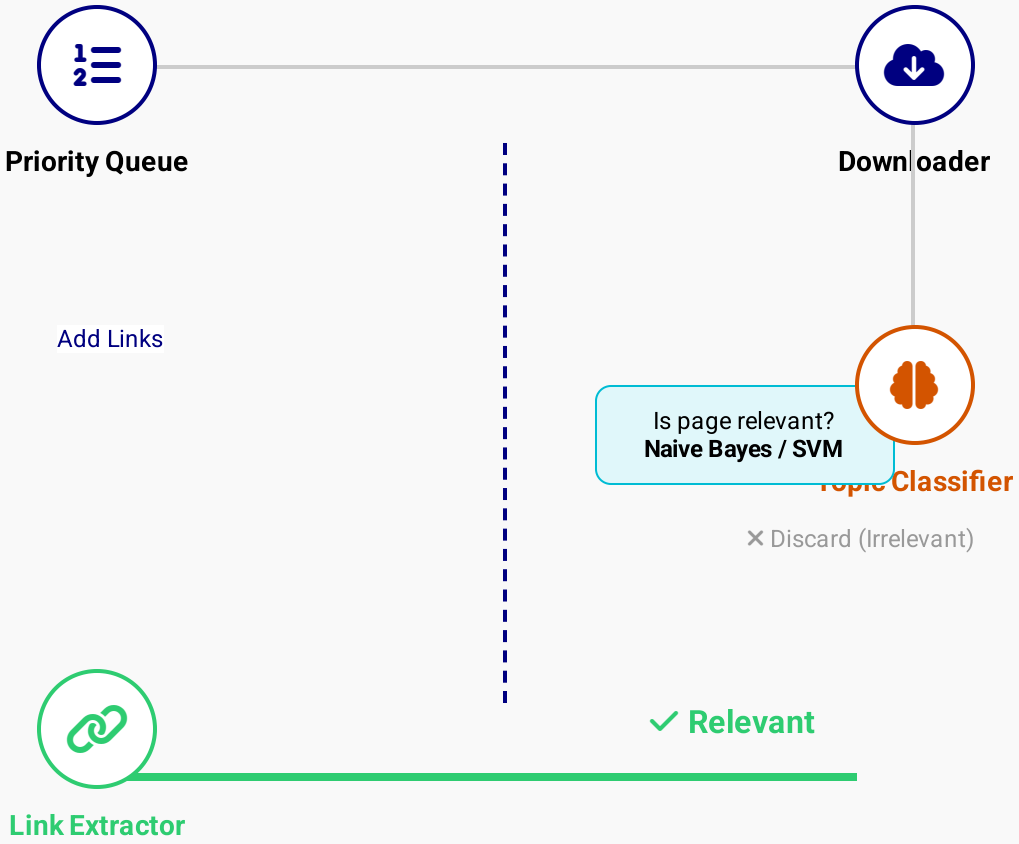
Estimated to be **500x larger** than the Surface Web.

The Challenge

Crawlers are "browsers without keyboards." They cannot easily fill out forms or log in.



Focused Crawling



The Goal

Selectively crawl pages relevant to a specific topic (e.g., "Health", "Sports") while ignoring the rest of the web.

The Challenge

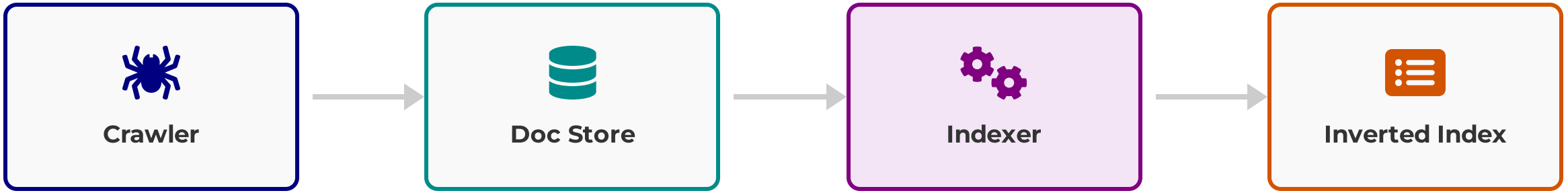
We must predict if a URL is relevant *before* downloading it.

Topic Locality

Assumption: Relevant pages tend to link to other relevant pages.

Strategy: Prioritize URLs found on highly relevant pages or with relevant anchor text.

Indexing Architecture



Fetches pages and extracts metadata.

Stores raw content & headers compressed.

`BigTable` `HBase`

Parses, tokenizes, and inverts the graph.

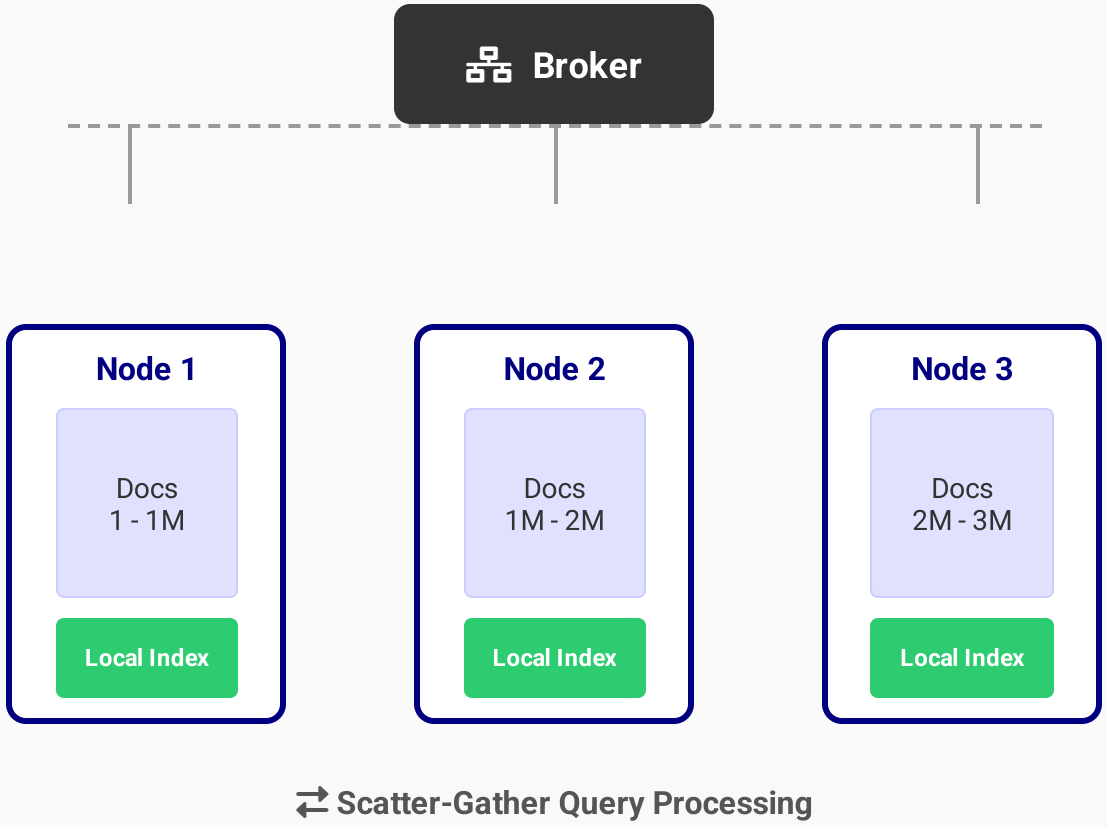
`MapReduce` `Spark`

Distributed lookup table (Term → Postings).

`Lucene` `Elastic`

Decoupled Architecture: Crawling and Indexing run asynchronously.

Document Partitioning (Local Indexing)



The Concept

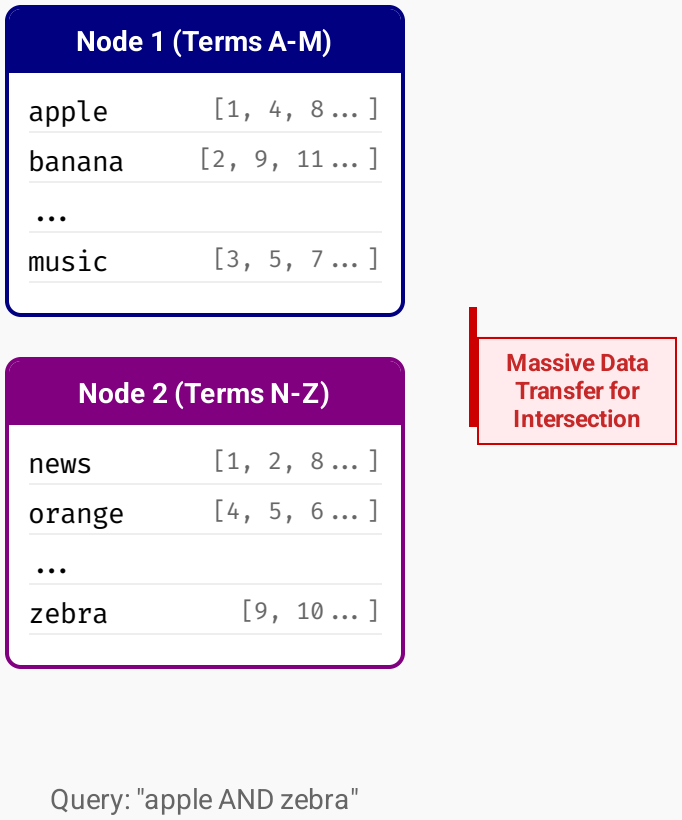
Each machine holds a subset of the documents and builds a **complete inverted index** for that subset.

Query Process

The broker broadcasts the query to **all** nodes. Each node returns its top k results. The broker merges them to find the global top k.

- **Scalable Updates:** Adding new documents is easy (just add a new node or append to existing ones).
- **High Load:** Every query must be processed by every machine ($O(N)$ total work).

Term Partitioning (Global Index)



Pros

- ✓ Single-term queries only need to contact one server.
- ✓ Easy to handle concurrency for simple lookups.

Cons

- × **Network Bottleneck:** Multi-word queries require sending large postings lists across the network to intersect.
- × **Load Imbalance:** Common terms ("the", "news") create hot spots that overload specific nodes.

Verdict: Rarely used in large-scale web search due to the high cost of intersection.

Dynamic Indexing

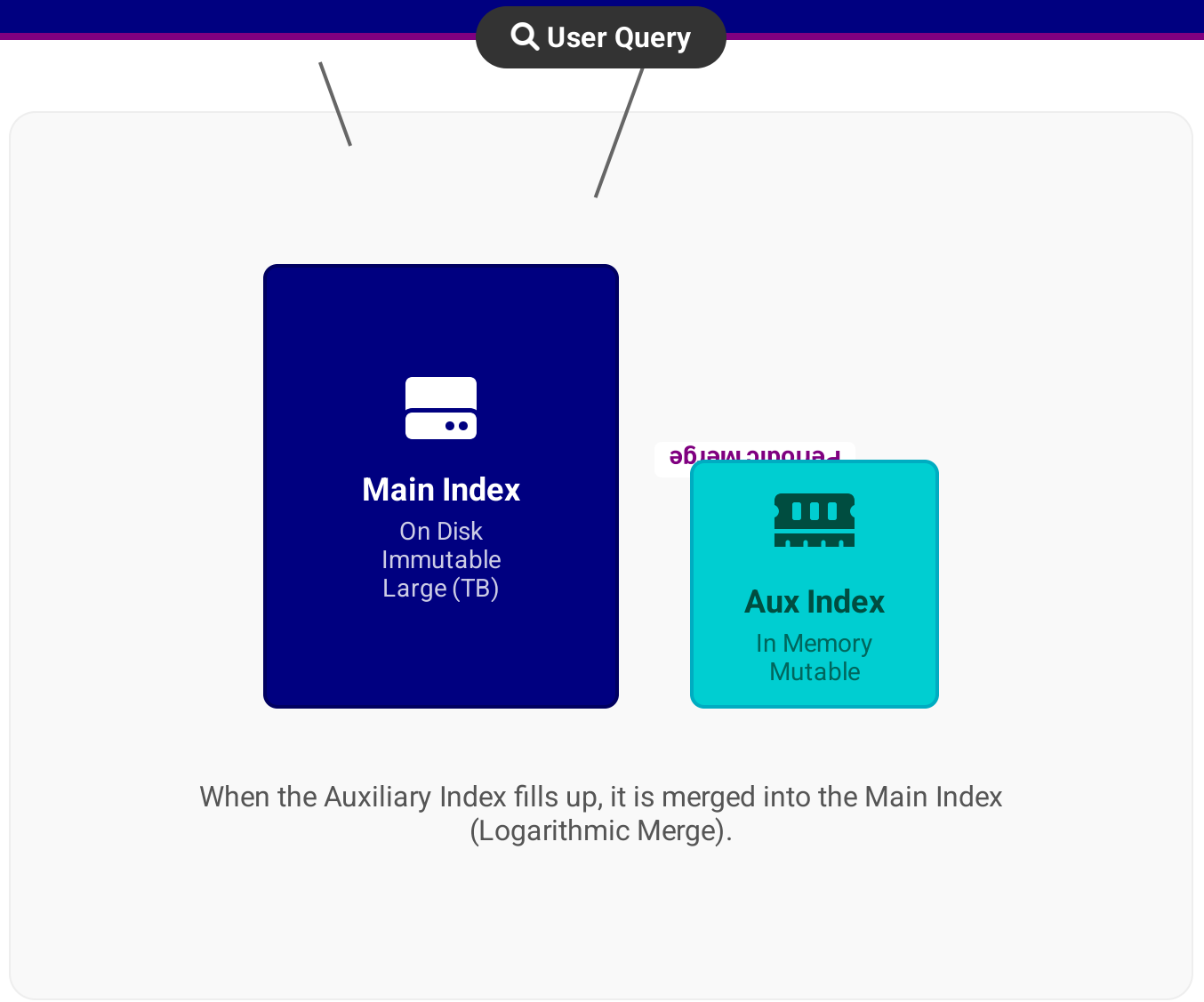
The Problem

Inverted indexes are highly compressed and sorted. Inserting a new document ID into the middle of a posting list requires rewriting the entire list.

The Solution

Maintain a small, mutable **Auxiliary Index** in memory for recent updates.

Search Process:
Query = (Main Index) U (Auxiliary Index)
Deletions are handled by a bit-vector filter.



Log-Structured Merge (LSM) Trees

1. Write to Memory

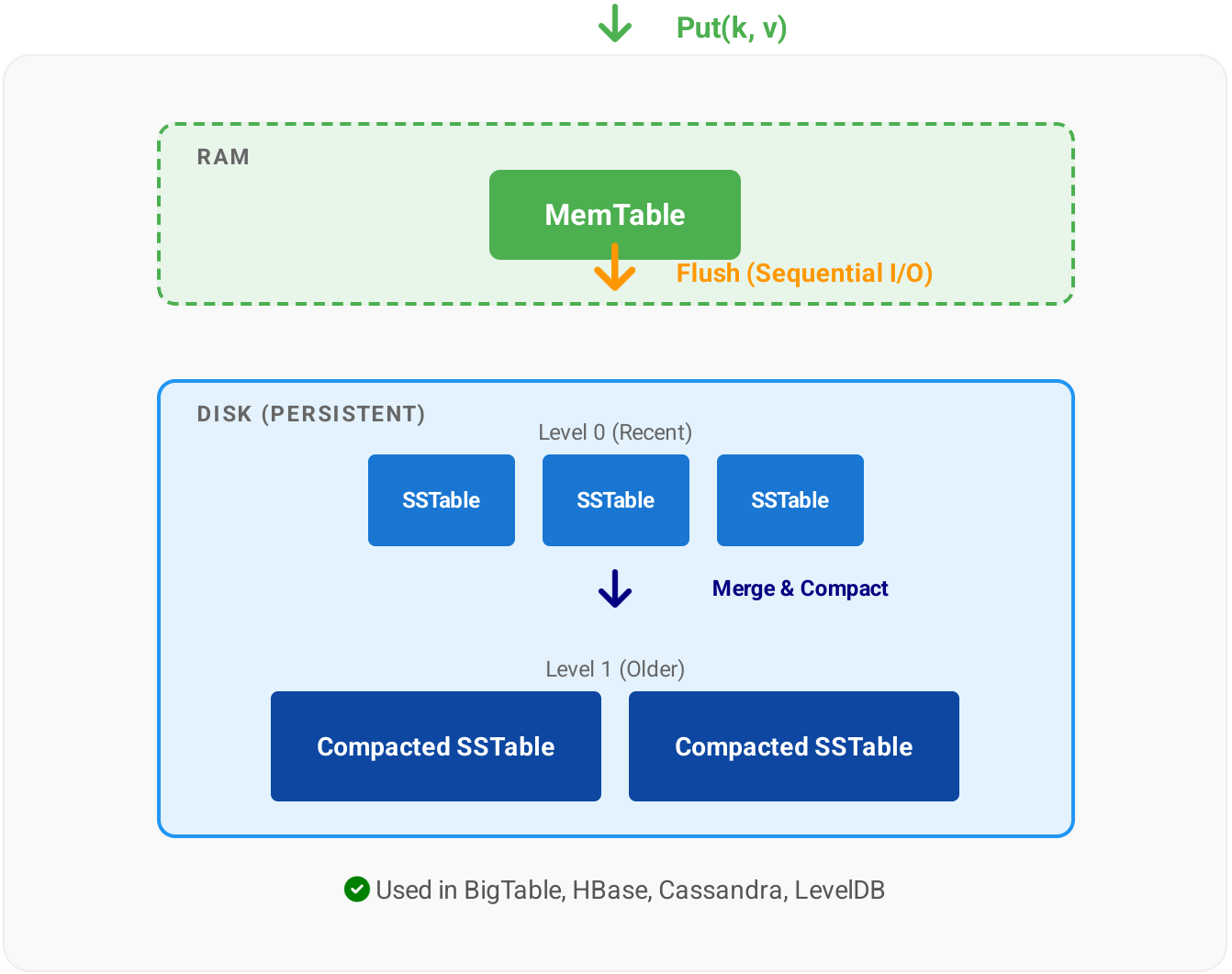
Updates are written to an in-memory **MemTable** (balanced tree). This is fast (RAM).

2. Flush to Disk

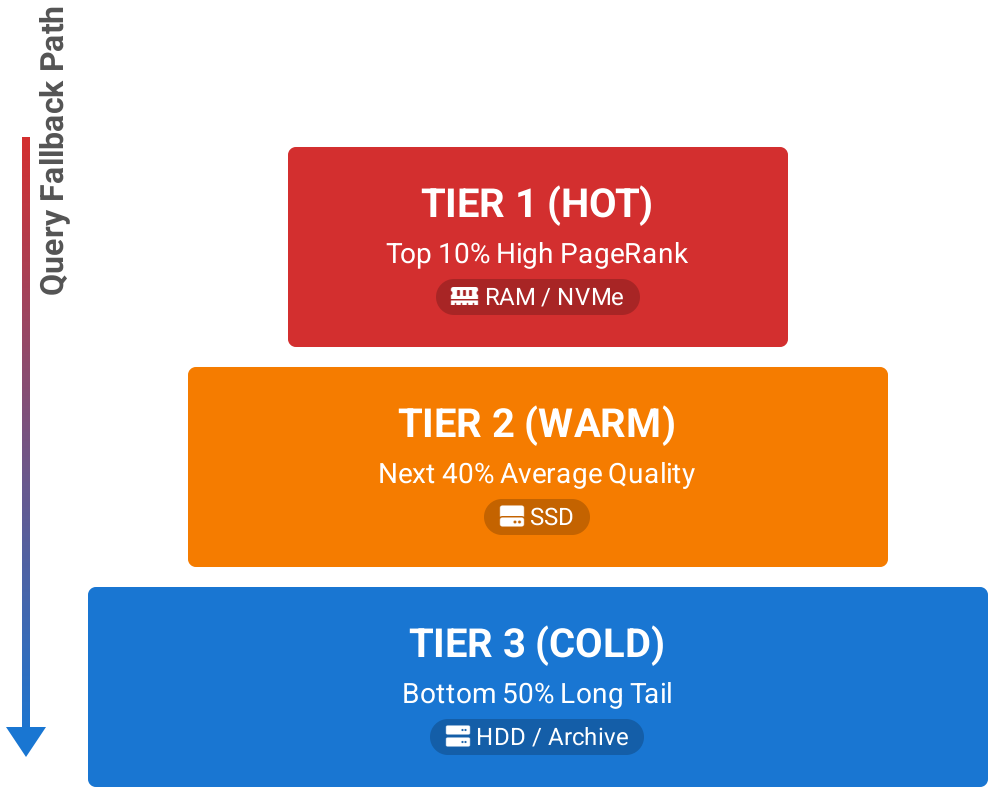
When MemTable is full, it is flushed to disk as an immutable **SSTable** (Sorted String Table). This is a sequential write.

3. Compaction

Background process merges multiple SSTables into larger ones, discarding deleted or overwritten keys.



Tiered Indexes



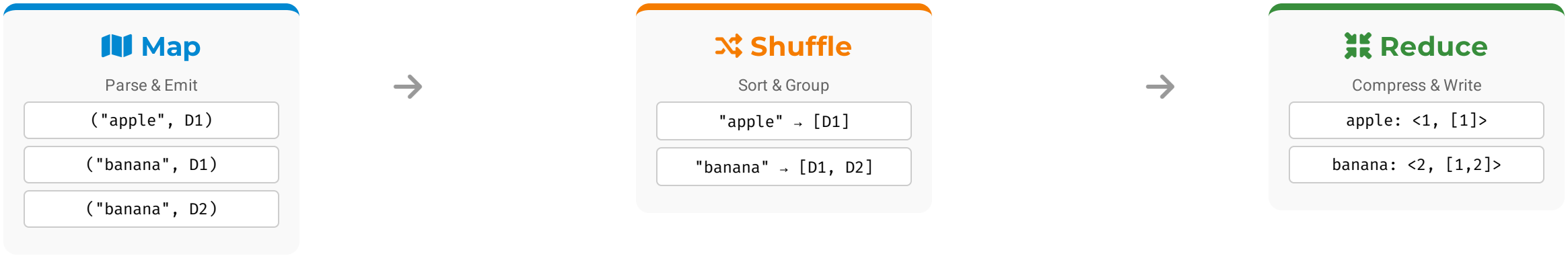
The Strategy

Don't search the entire web for every query. Most users are satisfied with results from the "best" pages.

1. **Query Tier 1:** If sufficient high-quality matches are found, return results immediately. (Fastest)
2. **Fallback:** If not enough results, query Tier 2.
3. **Last Resort:** Query Tier 3 only for obscure or specific queries (e.g., "1994 tax code section 5").

** Also known as "Champion Lists" or "Quality-Based Partitioning".*

MapReduce for Indexing (Recap)



Map Function

```
function map(docID, content) {
  for term in tokenize(content):
    emit(term, docID);
}
```

Reduce Function

```
function reduce(term, docList) {
  sortedDocs = sort(docList);
  postings = compress(sortedDocs);
  write(term, postings);
}
```

Key Transformation: MapReduce transforms the data from a **Document-Centric** view (Crawler) to a **Term-Centric** view (Index) efficiently at scale.

Fault Tolerance

Heartbeats

Worker nodes send periodic signals to the Master. If a signal is missed (timeout), the node is presumed dead.

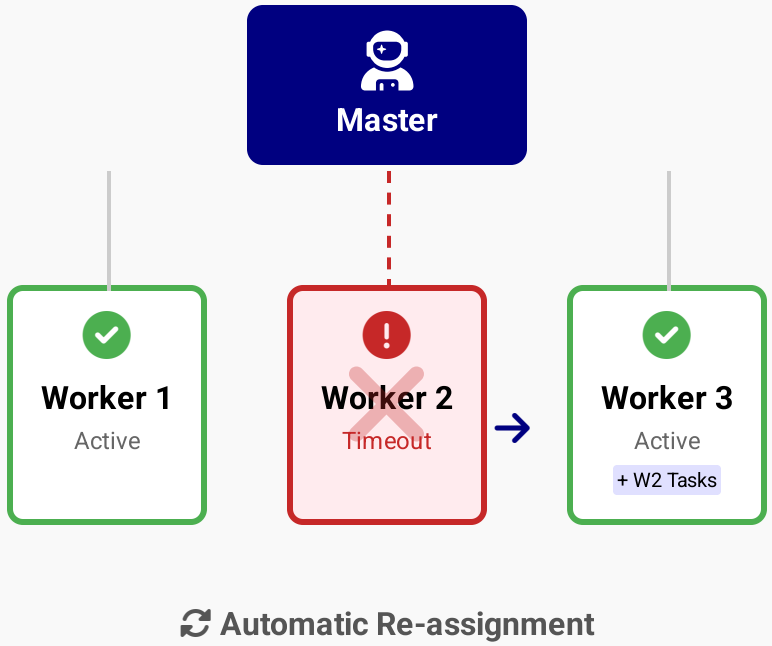
Checkpointing

Regularly save the state of the URL Frontier (visited URLs, queue status) to persistent storage (disk).

Replication

Store crawled data on distributed file systems (like HDFS) with 3x replication to prevent data loss.

> *"At scale, failure is the norm, not the exception."*



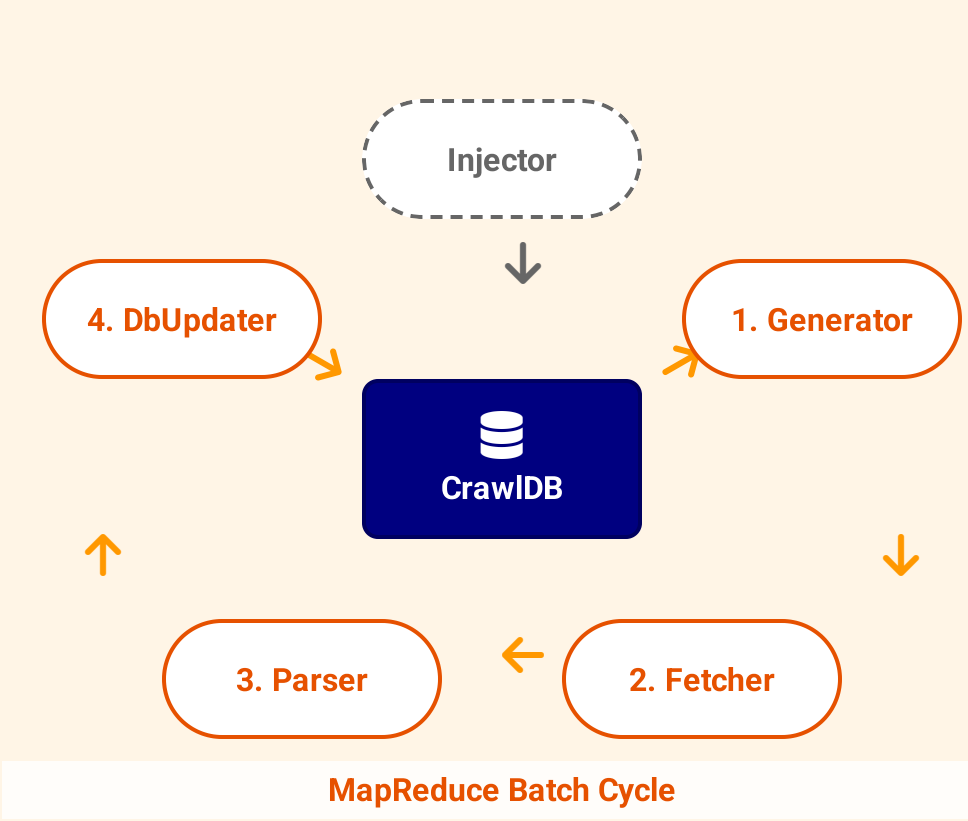
Case Study: Apache Nutch

The Origin of Hadoop

Nutch was created by Doug Cutting. To solve its scaling problems, he implemented MapReduce and HDFS, which later spun off as **Apache Hadoop**.

Key Data Structures:

- **CrawlDB:** The "Frontier". Stores URL, status, fetch time, retries.
- **LinkDB:** The Web Graph. Stores inlinks for PageRank.
- **Segments:** Batches of fetched content (raw HTML).



Lab: Introduction to Scrapy

What is Scrapy?

A fast, high-level web crawling and web scraping framework for Python.

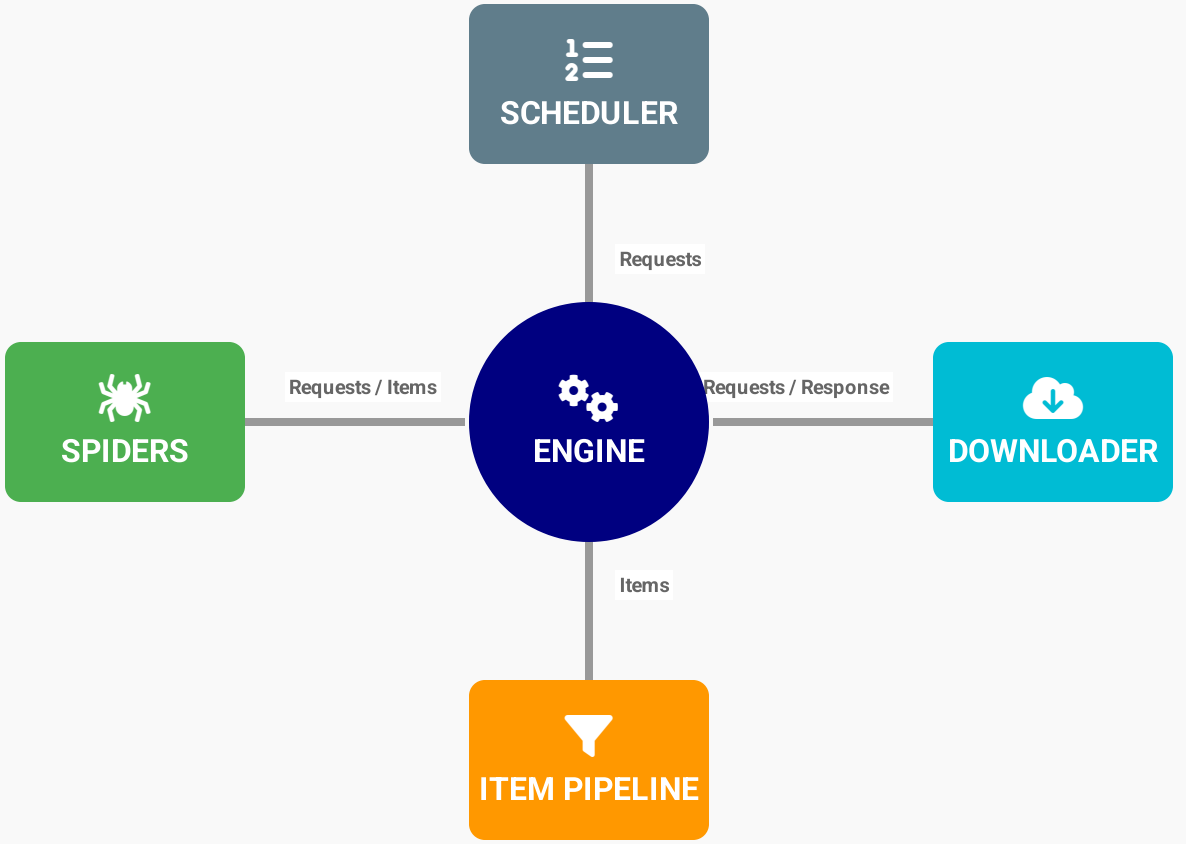
Asynchronous Core

Built on **Twisted**, an event-driven networking engine. It can handle thousands of concurrent requests without blocking.

Batteries Included

Comes with built-in support for:

- XPath & CSS Selectors
- JSON/CSV/XML Exports
- Robots.txt & User-Agent rotation



Data Flow Architecture

Scrapy Code Example

quotes_spider.py

```
import scrapy class QuotesSpider(scrapy.Spider): name = "quotes"
start_urls = [ 'http://quotes.toscrape.com/page/1/', ] def parse(self,
response): # Extract data using CSS selectors for quote in
response.css('div.quote'): yield { 'text':
quote.css('span.text::text').get(), 'author':
quote.css('small.author::text').get(), } # Follow pagination link
next_page = response.css('li.next a::attr(href)').get() if next_page
is not None: yield response.follow(next_page, callback=self.parse)
```

Class Definition

Spiders must subclass `scrapy.Spider` and define a unique `name`.

Entry Point

`start_urls` is a list of URLs where the spider begins crawling. The engine automatically generates requests for these.

Data Extraction

Use `response.css()` or `response.xpath()` to select elements. `yield` returns a dictionary as an Item.

Recursive Crawling

Find the "Next Page" link and `yield` a new Request, setting `self.parse` as the callback for the next response.

Summary: Crawling & Indexing

Crawling Basics

- **Politeness:** Respect Robots.txt and crawl delays.
- **Frontier:** Prioritize URLs (Mercator Model) based on quality and freshness.
- **DNS:** Caching is critical to avoid bottlenecks.

Distributed Systems

- **Consistent Hashing:** Distributes URLs to crawlers gracefully.
- **Fault Tolerance:** Checkpointing and replication handle node failures.
- **Decoupling:** Fetching and Indexing are asynchronous.

Duplicate Detection

- **Fingerprinting:** MD5/SHA1 for exact duplicates.
- **Shingling & MinHash:** Detect near-duplicates (30% of the web).
- **LSH:** Locality Sensitive Hashing for fast similarity search.

Indexing Strategy

- **Partitioning:** Document Partitioning (Local Index) is superior to Term Partitioning.
- **Dynamic Updates:** Use Auxiliary Indexes or LSM Trees.
- **MapReduce:** The standard for building indexes at scale.

Recommended Reading

Core Textbooks

Introduction to Information Retrieval

Manning, Raghavan, and Schütze (2008)

Read Chapter 19 & 20

Web Search Basics, Crawling, and Indexes

Search Engines: IR in Practice

Croft, Metzler, and Strohman (2010)

Read Chapter 9

Crawling and Feeds

Seminal Papers

- **Mercator: A Scalable, Extensible Web Crawler**
  Heydon & Najork (1999)
  The architecture used by AltaVista and early Google.
- **MapReduce: Simplified Data Processing on Large Clusters**
  Dean & Ghemawat (2004)
  The foundation of distributed indexing.
- **Parallel Crawlers**
  Cho & Garcia-Molina (2002)
  Analysis of partition strategies and politeness.
- **The Anatomy of a Large-Scale Hypertextual Web Search Engine**
  Brin & Page (1998)
  The original Google paper.

Tip: Focus on the *Mercator* paper for the exam, specifically the URL Frontier design.

Questions?

Thank You!

Feel free to ask about...

Crawler Architecture | URL Frontier | Distributed Indexing | PageRank | Spam Detection

Crawler Traps

What is a Trap?

A crawler trap is a set of URLs that cause a crawler to crawl indefinitely. This can be due to poor site design or intentional anti-bot measures.

The Impact

- Fills the URL Frontier with junk.
- Wastes bandwidth and storage.
- Prevents the crawler from discovering real content.

Common Examples

Calendar Trap

```
example.com/calendar/2026/feb
```

↓ Next Month

```
example.com/calendar/2026/mar
```

↓ Next Month

```
example.com/calendar/9999/dec
```

Infinite Directory

```
example.com/wiki/Apple/Banana/Apple/ ...
```

Dynamic Session IDs

```
example.com/page?sid=12345
```

```
example.com/page?sid=67890
```

URL Normalization (Canonicalization)

The Problem

Multiple distinct URL strings can refer to the exact same resource. Without normalization, the crawler will fetch the same page multiple times.

The Goal

Convert every URL into a standard **Canonical Form** before adding it to the Frontier or Index.

Case Normalization
`HTTP:// → http://`

Port Removal
`:80 (http) → [empty]`

Path Resolution
`/a/../b → /b`

Param Sorting
`?y=1&x=2 → ?x=2&y=1`

Fingerprinting (Exact Duplicates)

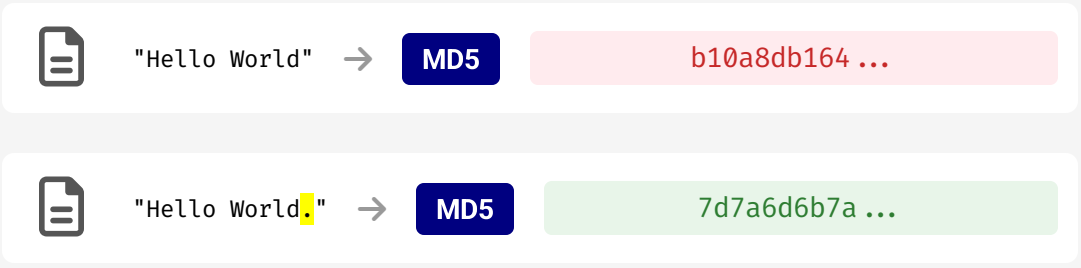
The Concept

Map the entire document content to a short, fixed-size bit string (e.g., 64-bit or 128-bit) using a cryptographic hash function.

Storage Efficiency

Instead of comparing full text (MBs), we compare hashes (Bytes).
O(1) Lookup: Check if hash exists in a Hash Table or Bloom Filter.

The Avalanche Effect



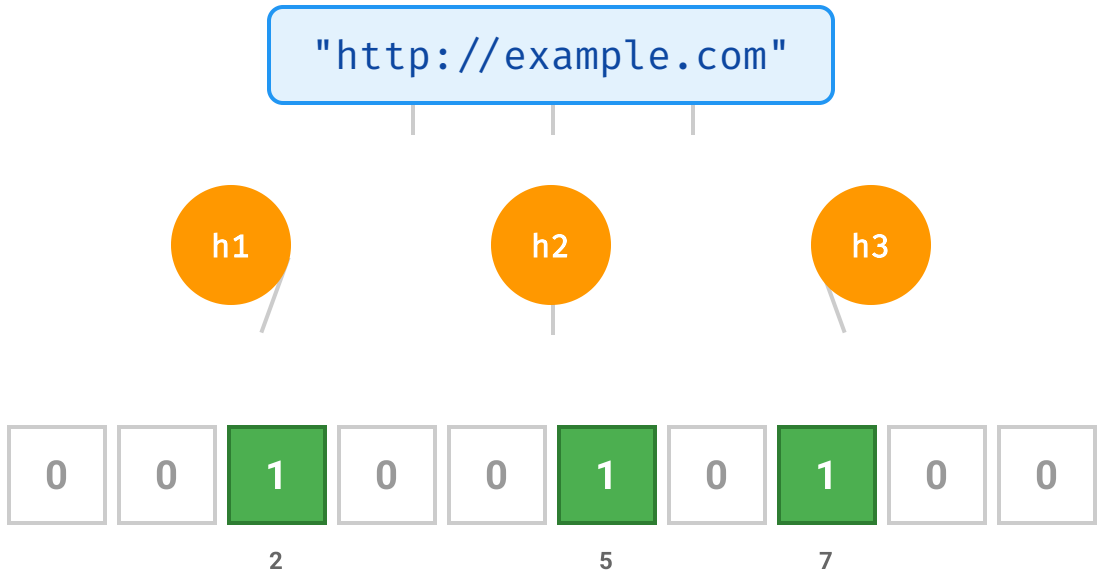
A single character change results in a completely different hash.

Algorithm	Bit Length	Collision Probability	Speed
MD5	128-bit	Very Low (1 in 3.4×10^{38})	Very Fast
SHA-1	160-bit	Extremely Low	Fast
SHA-256	256-bit	Negligible	Moderate

Limitation: Fingerprinting fails for *Near-Duplicates* (e.g., same article with a different timestamp or ad). For that, we need **Shingling**.

Bloom Filters

A space-efficient **probabilistic data structure** used to test whether an element is a member of a set.



No False Negatives

If the filter says "No", the URL has **definitely not** been visited. Safe to crawl.

False Positives

If it says "Yes", the URL *might* have been visited (or it's a hash collision).

Space Efficient

Uses ~10 bits per element regardless of URL length. (1 billion URLs ≈ 1.2 GB RAM).

Connectivity Server

The Purpose

A specialized service designed to store the **Web Graph** (who links to whom) efficiently to support link analysis algorithms like PageRank.

The Challenge

The graph is massive. Storing it as a simple list of pairs `(Source, Target)` is too large for RAM.

```
Nodes: ~50 Billion
Edges: ~1 Trillion
Raw Size: > 10 TB
```

Adjacency List (Raw)

Doc 1	`[5, 10, 15, 20, 25]`
Doc 2	`[5, 10, 16, 20, 25]`

↓ Compression

Compressed Storage

1. Gap Encoding (Deltas)

Doc 1	`[5, +5, +5, +5, +5]`

2. Reference Encoding (Similarity)

Doc 2	`Copy Doc 1` `Delete [15], Add [16]`

** Web pages on the same host often share 80%+ of their links (navigation menus). Reference encoding exploits this.*

Link Extraction & Parsing

</> The Messy Web

HTML on the web is often broken. Tags are unclosed, attributes are unquoted, and nesting is incorrect.

Where are the Links?

- `<a href="...">` (Standard)
- `<frame src="...">` (Legacy)
- `<link href="...">` (CSS/RSS)
- `onclick="window.location=..."` (JS)

DO NOT use Regex to parse HTML.
Use a robust parser (DOM).

Relative URL Resolution

Base URL	`http://example.com/blog/post.html`
Link Found	`../images/logo.png`
↓	
Absolute	`http://example.com/images/logo.png`

Beautiful Soup · lxml · Google Gumbo

Character Encoding Issues

The Tower of Babel

The web is a mix of legacy encodings: UTF-8 (98%), ISO-8859-1 (Western), Windows-1252, Shift_JIS (Japanese), GB2312 (Chinese).

Mojibake

Decoding bytes with the wrong charset results in garbled text.

Bytes	C3 A9
Latin-1	Ã©
UTF-8	é

Detection Strategy (Priority Order)

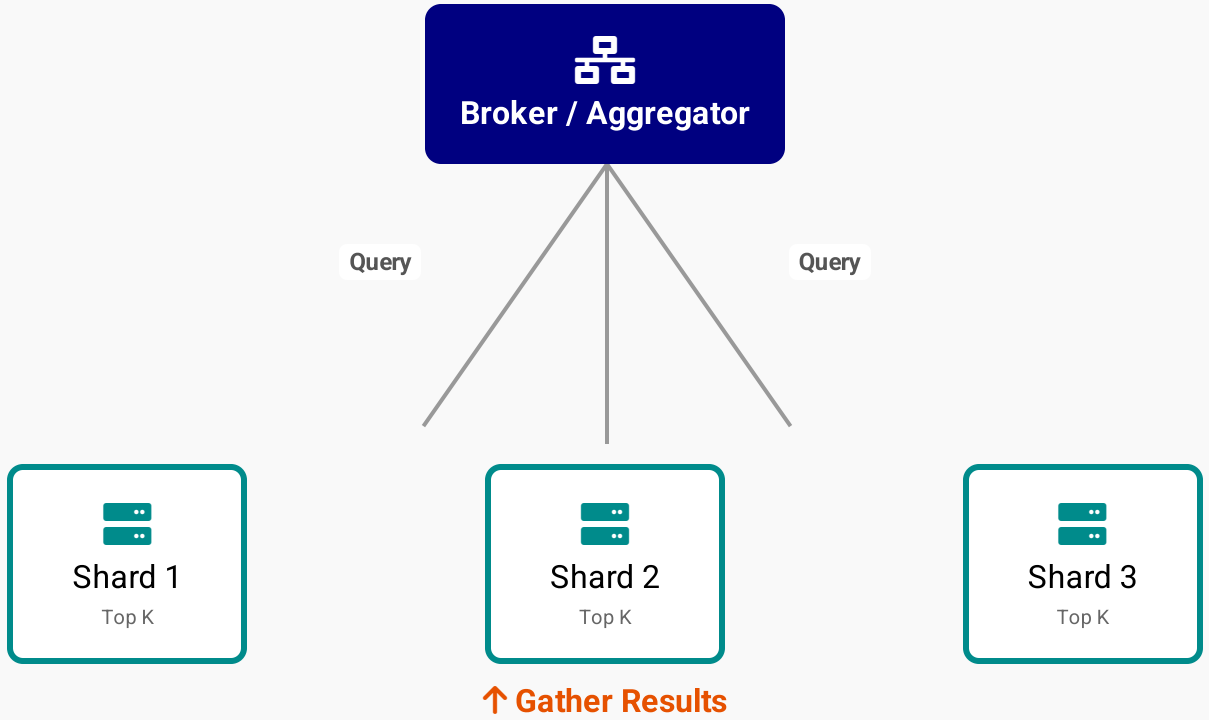
1. **HTTP Headers**
   `Content-Type: text/html; charset=utf-8`
   Often reliable, but sometimes misconfigured by servers.

2. **HTML Meta Tags**
   `<meta charset="utf-8">`
   Embedded in the document head.

3. **Heuristics (Chardet)**
   `chardet.detect(bytes)`
   Analyze byte frequency distribution to guess the language/encoding. Slow but necessary fallback.

Golden Rule: Convert EVERYTHING to UTF-8 internally.

Querying Distributed Indexes



1. Scatter

The Broker broadcasts the user query to **all** index shards (partitions) in parallel.

2. Local Search

Each shard executes the query on its local index and finds the **Top K** matching documents.

3. Gather & Merge

The Broker collects $N \times K$ results, merges them (sorting by score), and returns the final Top K to the user.

Tail Latency: The query is as slow as the *slowest* shard.

Caching Query Results

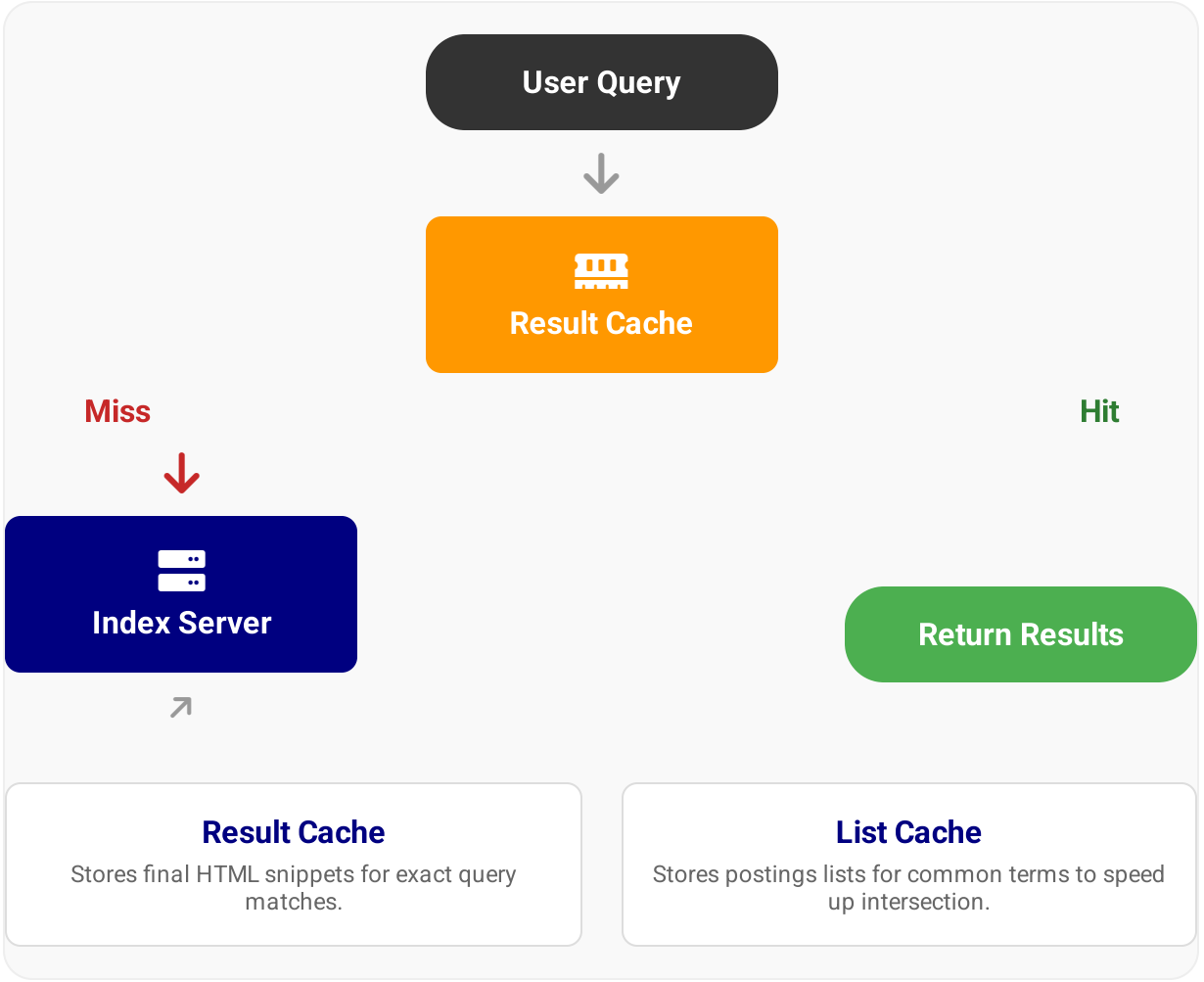
Why Cache?

Query processing is expensive (disk I/O, intersection, ranking).
Zipf's Law: A small number of queries (e.g., "facebook", "youtube", "weather") account for a huge volume of traffic.

Eviction Policy

Caches have limited size (RAM). We use **LRU (Least Recently Used)** to discard old entries when the cache is full.

Performance Impact:
Cache Hit: ~10ms
Cache Miss: ~200ms+



Index Compression Recap

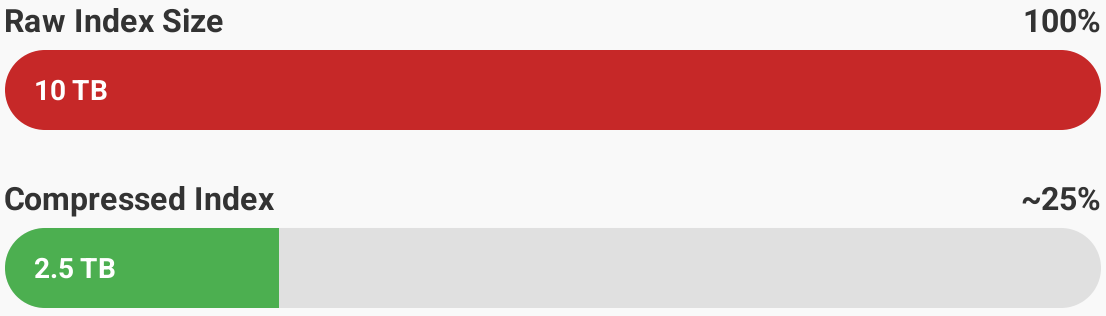
Why Compress?

It's not just about saving disk space. **Disk I/O is the bottleneck.**

The Trade-off

Decompressing data in the CPU is significantly faster than reading uncompressed data from the disk or network.

> **Cache Utilization:**
> A compressed index allows more of the posting lists to fit in the OS Page Cache (RAM), reducing disk seeks.



Key Techniques (Week 4)

Gap Encoding	`Store d-gaps (docID - prev)`
Variable Byte (V-Byte)	`7 bits payload, 1 bit flag`
Front Coding	`Compress Dictionary Terms`

Security & Privacy in Crawling

User Privacy

- **PII Redaction:** Automatically detect and mask SSNs, credit cards, and emails.
- **Right to be Forgotten:** Compliance with GDPR/CCPA removal requests.
- **Private Content:** Respect `Disallow: /private` in robots.txt.

Goal: Protect the User

Crawler Safety

- **Malware:** Crawling infected pages can compromise the fetcher nodes.
- **Honeypots:** Hidden links designed to trap and ban bots.
- **XSS/Injection:** Malicious scripts executing in the parser.

Goal: Protect the Infrastructure

Target Safety

- **DDoS Prevention:** Aggressive crawling can take down small servers.
- **Form Submission:** Avoid submitting forms that trigger actions (e.g., "Delete").
- **Politeness:** Adhere to `Crawl-Delay` directives.

Goal: Do No Harm

Legal Issues in Crawling

Copyright & Fair Use

Search engines rely on **Fair Use**. Copying content to create an index is considered "transformative" (it helps people find the original), provided you don't resell the content itself.

Trespass to Chattels

An old common law tort. If a crawler consumes so much bandwidth that it impairs the server's function, it can be sued for "trespassing" on the server hardware.

CFAA (Computer Fraud & Abuse Act)

Often used to prosecute "unauthorized access." The key question: Does violating a ToS count as "unauthorized access"?

Key Case Law

2000

eBay v. Bidder's Edge

Ruling: Injunction granted against Bidder's Edge. Their crawler was consuming capacity and ignoring requests to stop (Trespass to Chattels).

2006

Field v. Google

Ruling: Google's cache is Fair Use. If a site wants to opt-out, they must use standard protocols like `robots.txt` or `noarchive`.

2019

hiQ Labs v. LinkedIn

Ruling: Scraping *publicly available* data likely does not violate the CFAA, even if the site owner objects.

Future of Crawling

Mobile-First Indexing

Googlebot now primarily crawls as a smartphone user agent.
Sites must be responsive and fast on 4G networks.

Standard since 2020

Headless Rendering

The "App-ification" of the web (React, Vue) requires executing JavaScript to see content.
Cost: 100x more CPU than parsing HTML.

Puppeteer / Selenium

AI & LLM Training

Crawlers are no longer just for search. They feed data to Large Language Models.
New controls: `Google-Extended` in robots.txt.

The Web as Dataset

The Engineering Challenge: As the web becomes more dynamic and app-like, the computational cost of crawling is increasing exponentially.

Rendering JavaScript

</> The SPA Problem

Single Page Applications (React, Vue) load content dynamically. A standard crawler only sees the initial shell.

view-source:example.com

```
<body>
  <div id="root"></div>
  <script src="app.js"></script>
  <!-- No content here! -->
</body>
```

Crawler sees nothing.

Headless Browsers

Run a full browser instance (Chrome/Firefox) without a UI to execute JS and extract the DOM.

Puppeteer

Google's Node.js library for controlling Chrome.

Playwright

Microsoft's modern automation tool (Cross-browser).

Selenium

The legacy standard. Supports many languages.

Performance Cost

Rendering is **10x-100x slower** than simple HTML parsing. It requires significantly more CPU and RAM.

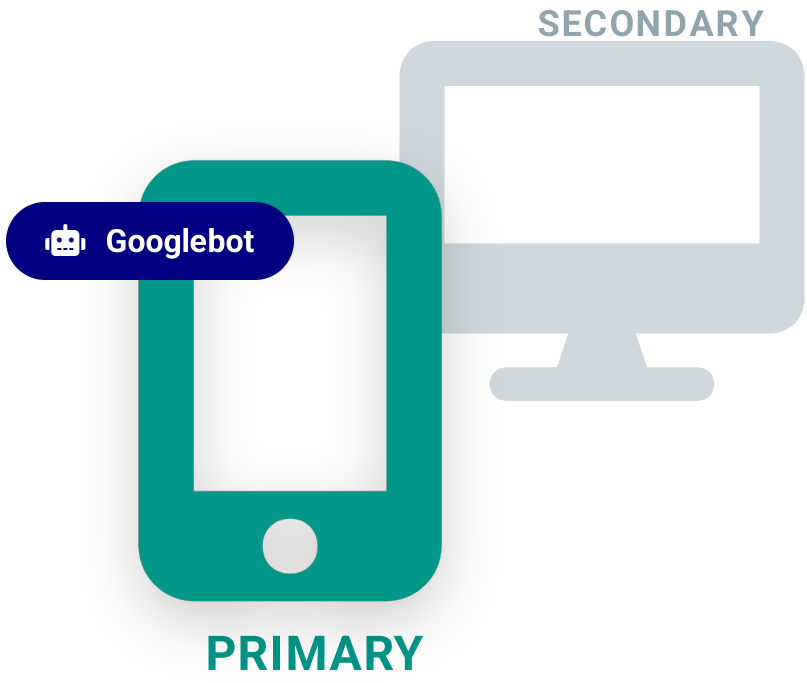
Mobile-First Indexing

The Paradigm Shift

Historically, Google indexed the **Desktop** version of a page. Since ~2019, Google predominantly uses the **Mobile** version for indexing and ranking.

Key Implications

- **Content Parity:** Ensure important text/links on desktop are also visible on mobile.
- **Hidden Content:** Text in accordions or tabs (for UX) is now treated as fully visible/weighty.




```
User-Agent: Mozilla/5.0 (Linux; Android 6.0.1; Nexus 5X ... )
```

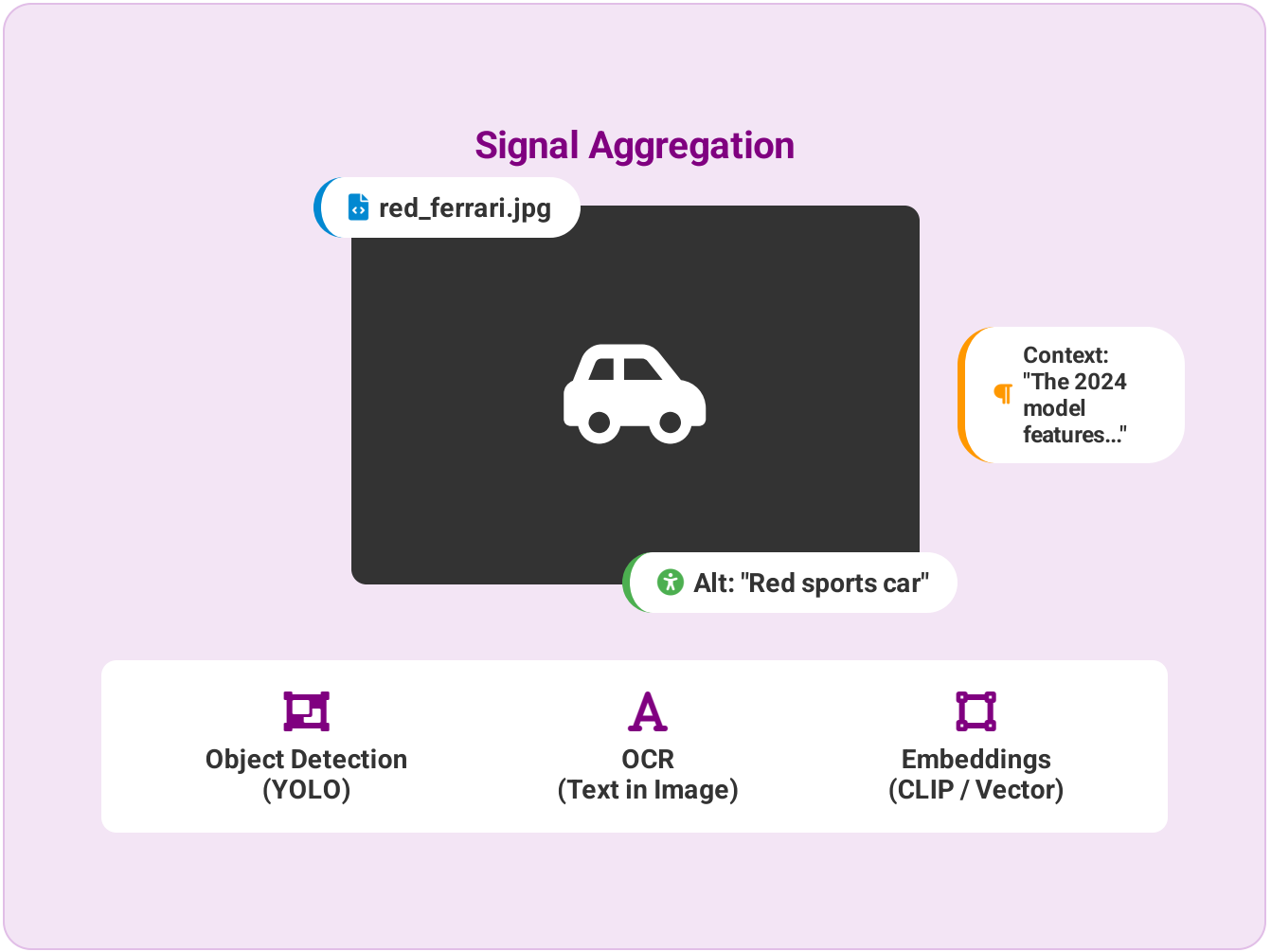
Crawling for Visual Search

Context Extraction

Search engines rely heavily on text signals **surrounding** the image to understand its content.

Content Analysis

Modern crawlers download the image pixels to run Computer Vision models (Deep Learning) for tagging and embeddings.



Video Indexing

The Black Box

To a raw crawler, a video file is just a massive binary blob. It has no inherent text to index.

Deep Search

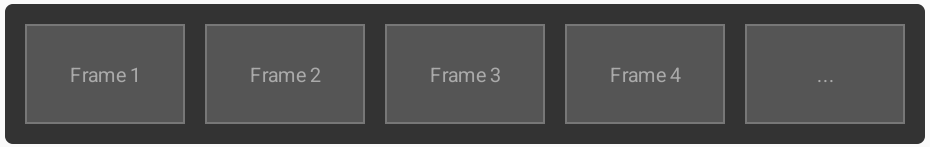
Users don't just want to find a video; they want to find the **exact moment** (e.g., "Jump to 4:20") where a topic is discussed.

Extraction Pipeline

Metadata (Weak Signal)
Title, Description, Tags. Often spammy or incomplete.

ASR (Speech-to-Text)
Automated transcription (e.g., Whisper) converts audio tracks to searchable text.

Visual Analysis (OCR & CV)
Extract text from slides/chyrons. Detect objects (e.g., "Cat", "Car") in keyframes.



Real-Time Search

The Need for Speed

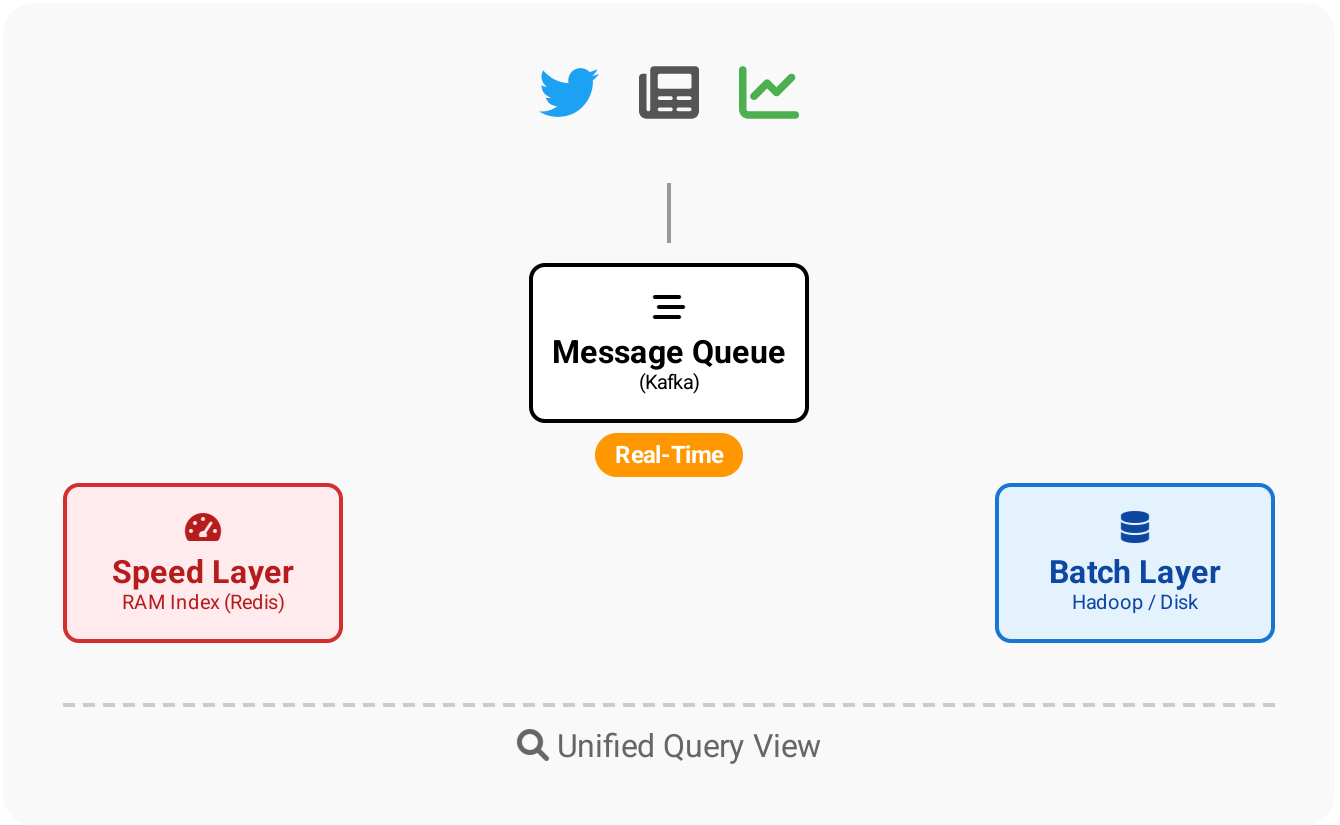
Traditional crawling (batch) takes days. Breaking news ("Earthquake", "Super Bowl") must be indexed in **seconds**.

Push vs. Pull

Old: Polling RSS feeds every 5 minutes.
New: Firehose APIs (Twitter/X, GNIP) push data instantly to the engine.

⚡ **QDF (Query Deserves Freshness):**
Algorithm detects "hot" topics and boosts recent content.

Lambda Architecture



Final Thoughts

**"Crawling is not just about downloading pages.
It is about managing infinite scale with finite resources."**

Engineering Feat

A distributed systems challenge requiring fault tolerance, DNS caching, and massive deduplication.

Social Contract

Politeness is paramount. We are guests on other people's servers. Respect `robots.txt`.

Moving Target

The web evolves (HTML → JS → Apps). Crawlers must adapt to render content and avoid traps.

Garbage In, Garbage Out: The Crawler defines the Index quality.